

DAW1°

RETO 3: MONTANDO EL PUZZLE

Integrantes: Aitor Martin, Luka Ruiz, Marcos
Movilla, Aitor Garcia, Ainur Cano

Duración: 11 semanas



Índice de contenido

Introducción	4
Programación	5
Introducción	5
2. – CODIGO DE CONEXIÓN A LA BASE DE DATOS	6
Resto de la documentación de PRG	10
Consultar Libro (SELECT):	22
Insertar Libro (INSERT):	23
Modificar Libro (UPDATE) :	23
Borrar Libro (DELETE) :	24
Lenguaje de marcas	25
1. Transformación de tablas SQL a XML	26
2. XSLT	28
3. Consultas XPATH sencillas	30
4. Consultas anidadas	32
5. Consultas con funciones	34
6. Consultas anidadas + funciones	36
7. Consultas XQUERY explicación	38
¿Qué es XQuery?	38
Aplicación usada:	39
Como funciona	39
8. Consultas XQUERY realizadas	41
Bases de datos	46
Diagrama E/R de nuestra biblioteca	47
3 – OPERACIONES: CONSULTAS, MODIFICACIONES, BORRADO E INSERCCIONES	48
TABLA LIBROS	48
TABLA AUTOR	49
TABLA USUARIOS	50
TABLA EJEMPLAR	51
TABLA PRESTAMOS	52
Apartado de Lenguaje de Marcas	54
• Similitudes entre XQuery y SQL	54
• ¿XQuery se parece más a SQL o a PL/SQL?	55
Ejercicios de ejemplos	57



Conclusión	78
Entornos de desarrollo	79
Diagrama de clases.....	80
Explicación	80
METODOS	81
• INSTANCIAS DE LAS CLASES.....	84
Diagrama de casos de uso	92
Explicación	93
Diagrama de secuencia	94
Explicación	95
Diagrama de objetos	96
Explicación	97
Enlace al sistema de versiones.....	98



Introducción

La Biblioteca está llevando a cabo una actualización completa de su sistema de gestión con el objetivo de modernizar sus procesos y mejorar el acceso a la información. Para ello, incorporaremos nuevas funcionalidades basadas en tecnologías actuales: uso de XML para el intercambio de datos, validación mediante DTD/XML Schema, consultas con XPath y transformaciones XSLT para generar otros formatos.

Además, se desarrollará una **aplicación web** para gestionar libros y autores desde formularios, y una **aplicación Java por consola** para controlar los préstamos, ejemplares, estados, retrasos y sanciones, conectada a un SGBD (**Sistema Gestor de Bases de Datos**).

El proyecto también integra criterios de **sostenibilidad digital**, reduciendo el uso de papel y optimizando recursos, junto con un análisis de **transformación digital** para mejorar la eficiencia de la biblioteca. Finalmente, se revisará la documentación laboral de la empresa para garantizar un funcionamiento interno más organizado y actualizado.

Programación

Introducción

Como biblioteca, hemos identificado la necesidad de modernizar y optimizar la gestión de nuestros recursos, usuarios y procesos internos. Para dar respuesta a estas necesidades, hemos desarrollado una **aplicación completa orientada a objetos**, diseñada específicamente para adaptarse a nuestro funcionamiento real y a las tareas que nuestro personal realiza diariamente.

Asimismo, la incorporación de tests unitarios mejora la calidad del proyecto, ya que permiten comprobar cada componente por separado, detectar errores antes y asegurar que futuras mejoras no generen fallos.

En conjunto, esta aplicación es una solución creada específicamente para nuestra biblioteca, ajustada a nuestras necesidades y preparada para seguir evolucionando.

Además, al final de esta página encontraréis un enlace al **vídeo explicativo de la aplicación**, donde se muestra su funcionamiento paso a paso, junto con el **código completo del proyecto**, disponible para su consulta y revisión.



2. – CODIGO DE CONEXIÓN A LA BASE DE DATOS

Este código en Java tiene como objetivo intentar establecer una conexión a una base de datos MySQL y verificar si fue exitosa.

```
1 public class testConexion {
2
3     public static void main(String[] args) {
4
5         String hostname = "localhost";
6         String database = "biblioteca";
7         String username = "root";
8         String password = "";
9         Connection cnx = null;
10
11         String url = "jdbc:mysql://" + hostname + ":3306/" + database + "?useSSL=false";
12
13         cnx = InputOutput.getConnection(url, username, password);
14         if (cnx == null) {
15
16             System.out.println("Sin conexion a la base de datos :(");
17             return;
18         }
19         System.out.println("Conexion a la base de datos exitosa !!!");
20     }
21 }
22 }
```

1.1 Configuración: Para empezar, se definen 4 variables: la primera indica donde tenemos alojada la base de datos. La segunda es el nombre de la base de datos a la que se desea conectar. La tercera el usuario de MYSQL y por último, la cuarta será la contraseña de la base de datos.

1.2 Desarrollo de la Conexión: Se crea una variable llamada cnx (que es fundamental) para almacenar la conexión más adelante. Empieza vacía (valor null).

1.3 Intento de Conexión: Se llama a un método de otra clase llamado InputOutput. Este método, recibe la URL, el usuario y la contraseña para intentar tener conexión a la base de datos. Si se logra tener conexión, se guarda la conexión en cnx.

1.4 Verificación: Se usa una condicional if para saber qué ha sucedido:

- Si la conexión ha fallado (if (cnx == null)) significa que el método no pudo conectarse. Se muestra el mensaje "Sin conexión a la base de datos".

- Si la conexión ha sido exitosa, el código se salta la condición anterior y pasa directamente a mostrar el mensaje "Conexión a la base de datos exitosa !!!".

```
1 public class InputOutput {
2
3     public static Connection getConexion(String u, String n, String p) {
4
5         try {
6             Connection conn = null;
7             conn = DriverManager.getConnection(u, n, p);
8             return conn;
9         } catch (SQLException e) {
10             e.printStackTrace();
11             return null;
12         }
13     }
14 }
15
```

Con este código, recogemos los parámetros de la clase anterior en el que contiene el método que realmente hace el trabajo de conectar a la base de datos.

Es el que realmente sabe cómo verificar los datos. Si la contraseña es correcta, devuelve la conexión, en caso contrario si falla o los datos están mal, anota el error y devuelve un valor null, indicando que no se ha podido conectar a la base de datos.

1.5 Cabecera: - public static: Significa que el método se puede usar desde cualquier otra clase.

- Connection: Tipo de dato que va a devolver este método.
- (String u, String n, String p): Los parámetros que recibe desde el otro archivo.

1.6 Bloque try – catch: Muchas cosas pueden salir mal y por ello, Java obliga a proteger este código usando un try-catch.

- Connection conn = null;; Se inicializa una variable de conexión vacía.
- conn = DriverManager.getConnection(u, n, p); Utiliza el conector para intentar abrir la sesión en la base de datos con los parámetros recibidos.
- return conn;; Si funciona sin error, el método devuelve la conexión.



1.7 Bloque catch: Si algo falla en el try, Java interrumpe la ejecución y captura el error en el bloque catch:

- SQLException e: Captura el error específico de bases de datos.
- e.printStackTrace();: Muestra por pantalla el error de por qué falló
- return null;; Como no se ha podido conectar, el método devuelve un valor vacío.

Funcionamiento de la Aplicación

La aplicación funciona mediante **menús en consola**, donde el usuario navega escribiendo el número de la opción que desea ejecutar. Todo el sistema gira alrededor de cuatro operaciones principales: **Consultar, Insertar, Actualizar y Eliminar** datos de la base de datos.

1. Menú Principal

Al iniciar la app, aparece un menú con estas opciones:

- **Consultar**
- **Insertar**
- **Actualizar**
- **Eliminar**
- **Salir**

El usuario escribe un número del 1 al 5 para elegir qué acción quiere realizar.

```
=====
MENU DE LA BASE DE DATOS
=====
1. Consultar
2. Insertar
3. Actualizar
4. Eliminar
5. Salir
Seleccione una opcion:
```

2. Submenús por Categoría

Si el usuario elige **Consultar**, se abre otro menú con las entidades disponibles:

- Libros
- Autores
- Usuarios

- Ejemplares
- Préstamos

Cada opción muestra la información almacenada en esa categoría.

```
-- CONSULTAR --
1. Libros
2. Autores
3. Usuarios
4. Ejemplares
5. Prestamos
6. Volver
seleccione una opcion: █
```

3. Consultas

Cuando se selecciona una categoría (por ejemplo, *Libros*), la aplicación muestra todos los registros existentes, uno por línea, con sus campos principales: ID, título, ISBN, editorial, género y copias.

El sistema solo **lee y muestra** la información, sin modificarla.

```
===== CONSULTA DE LIBROS =====
=====
ID: 11 | Título: Crónica de una muerte | ISBN: 9788439734130 | Editorial: Random House | Género: Realismo mágico | Copias: 7
ID: 12 | Título: Fahrenheit 451 | ISBN: 9788490325070 | Editorial: Debolsillo | Género: Distopía | Copias: 9
ID: 13 | Título: El resplandor | ISBN: 9788497593748 | Editorial: Debolsillo | Género: Terror | Copias: 5
ID: 14 | Título: Sapiens | ISBN: 9788499926223 | Editorial: Debate | Género: Ensayo | Copias: 25
ID: 15 | Título: Dune | ISBN: 9788497596824 | Editorial: Debolsillo | Género: Ciencia ficción | Copias: 11
ID: 1233 | Título: hola | ISBN: 1232 | Editorial: hola1 | Género: terror | Copias: 12345
```

4. Insertar

Permite agregar nuevos registros.

El sistema pide al usuario los datos necesarios (por ejemplo, título, ISBN, editorial, etc.) y los guarda en la base de datos.

5. Actualizar

Permite modificar un registro existente.

El usuario selecciona el ID del elemento que quiere cambiar y luego introduce los nuevos valores.

6. Eliminar

El usuario escribe el ID del registro que desea borrar y la aplicación lo elimina de la base de datos.

7. Salir

Finaliza la ejecución del programa.



Resto de la documentación de PRG

1. – INTRODUCCION

En este reto de base de datos se explica el funcionamiento de la base de datos utilizada para la gestión de una biblioteca, con las asignaturas de Lenguaje de Marcas y Programación. En él, se explica cómo se diseña, conecta y manejar una base de datos, utilizando tanto el desarrollo web como la programación orientada a objetos.

En el primer apartado, se explica la estructura de la base de datos mediante el **modelo entidad-relación**, mostrando las tablas, sus campos y las relaciones entre ellas mediante claves primarias y claves ajenas. Todo esto, permite gestionar la información de la biblioteca organizadamente.

En el siguiente apartado, se muestra la **relación de la base de datos con la asignatura de Lenguaje de Marcas** explicando cómo la base de datos se conecta y como se utiliza desde una página web desarrollada en PHP utilizando también operaciones CRUD. Para comprender mejor cada parte del código, este, va acompañado de su instrucción SQL correspondiente.

Para terminar, en el último apartado, se muestra la relación de la base de datos con **la asignatura de Programación**, utilizando una aplicación orientada a objetos conectada a MySQL.

Se explica un ejemplo práctico de operaciones CRUD (Create, Read, Update y Delete) las cuales se han aplicado sobre las tablas de la base de datos.

2.- DIAGRAMA ENTIDAD – RELACION DE LA BASE DE DATOS CON TODOS LOS CAMPOS

En este apartado se explica la relación de la base de datos utilizada con todos los campos, indicando claves primaria y claves ajenas.

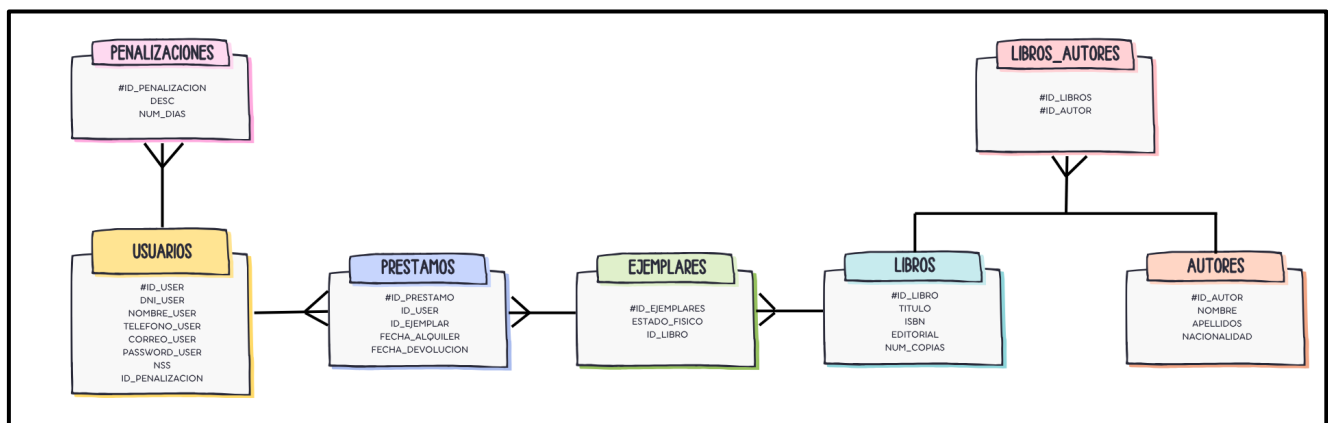


Tabla PENALIZACIONES:

La tabla PENALIZACIONES almacena las sanciones o penalizaciones que pueden recibir los usuarios de la biblioteca que incumplen las normas de entrega cuando incumplen alguna norma, por ejemplo: retraso en la devolución de libros, deterioro de ejemplares, o pérdida de material.

Cada penalización tiene un identificador único y un número determinado de días de sanción

Tabla USUARIOS

La tabla USUARIOS almacena toda la información personal y de acceso de los usuarios registrados en la biblioteca.

Cada usuario puede solicitar préstamos, tener sanciones, o realizar devoluciones de libros.

Relación de la tabla penalizaciones con la tabla usuarios:

Un usuario puede tener una penalización y una penalización puede estar asociada a varios usuarios.



Tabla LIBROS

La tabla LIBROS almacena la información general de los libros, registrando los datos bibliográficos de las obras disponibles.

Tabla EJEMPLARES

La tabla EJEMPLARES representa cada copia física de un libro.

Relación de la tabla libros con la tabla ejemplares:

Un libro puede tener muchos ejemplares y cada ejemplar pertenece a un único libro.

Tabla PRESTAMOS

La tabla PRESTAMOS registra los préstamos realizados por los usuarios de la biblioteca. Aquí se almacena: Qué usuario ha tomado prestado un libro, qué ejemplar fue prestado y las fechas tanto de préstamo como de devolución.

Relación de la tabla prestamos con la tabla usuarios:

Un usuario puede realizar muchos préstamos y un ejemplar puede ser prestado muchas veces a lo largo del tiempo

Tabla AUTORES

La tabla AUTORES contiene la información de los autores.

Tabla LIBROS_AUTORES

La tabla LIBROS_AUTORES es una tabla intermedia para la relación muchos a muchos entre libros y autores.

Relación de la tabla libros con la tabla autores:

Un libro puede tener varios autores y un autor puede escribir varios libros.

3.- BASE DE DATOS CON LA ASIGNATURA DE LLMM

En este apartado se explica que código de Lenguaje de Marcas afecta a la base de datos (conexión, consultas, modificaciones, inserciones, y borrados).



Para ello se ha trabajado con la pagina web de la biblioteca desarrollada en PHP, en la cual primero se ha creado una conexión a la base de datos y una vez realizada la base de datos, se han realizado algunos ejemplos de dichas operaciones hacia ella.

Tambien se explica la instrucción sql que se asemeja a dicho codigo.

CODIGO DE CONEXION A LA BASE DE DATOS:

```
1  <?php
2  session_start();
3  if (!isset($_SESSION['id_user'])) {
4      header("Location: login.php");
5      exit;
6  }
7
8  $conexion = new mysqli("localhost", "root", "", "biblioteca");
9  if ($conexion->connect_error) {
10     die("Error de conexión");
11 }
```

El código en PHP realiza dos puntos importantes: el primero es el control de acceso mediante sesiones y el segundo es la conexión a una base de datos MySQL.

Control de acceso:

`session_start();`: Inicia una nueva sesión para poder usar variables de sesión.

`if (!isset($_SESSION['id_user']))`: Esto sirve para comprobar si el usuario ha iniciado sesión. Comprueba si no existe la variable de sesión llamada 'id_user'.

Conexión a la base de datos:

`$conexion = new mysqli(...)`: Variable la cual intenta abrir una nueva conexión con la base de datos, recibiendo un objeto de tipo mysqli pasando 4 parámetros, los cuales son:

"localhost": El servidor donde tenemos la base de datos.



"root": El nombre de usuario predeterminado de MySQL.

"": La contraseña vacía.

"biblioteca": El nombre de la base de datos a la que nos conectamos.

A continuación comprobamos que la conexión sea exitosa:

- if (\$conexion->connect_error): Verifica si ocurrió algún error durante el intento de conexión.
- die("Error de conexión"); Si la conexión falla, mostramos el mensaje de error en la pantalla y termina la ejecución del programa.

EJEMPLOS DE DIFERENTES TABLAS CON OPERACIONES CRUD

TABLA USUARIOS: SELECT

```
1 $user = $conexion->query("SELECT ID_PENALIZACION FROM USUARIOS WHERE ID_USER = $id_user");
```

Este código realiza una consulta a la base de datos para obtener la penalización de un usuario en concreto.

\$user = ...: Variable que almacena el resultado de la consulta en la variable \$user pasando la conexión mediante una query.

\$conexion->query(...): Ejecutamos la consulta SQL utilizando el objeto de conexión creado anteriormente.

Indicamos que solo queremos obtener el dato de la columna de penalizaciones de la tabla llamada USUARIOS. Filtramos la búsqueda para que solo nos devuelva el registro del usuario cuyo id sea el mismo que la variable \$id_user.

***Esta instrucción se asemeja a la siguiente select en bases de datos: ***

SELECT ID_PENALIZACIONES FROM USUARIOS WHERE ID_USER = 1;

TABLA EJEMPLARES: SELECT



```
1
2 $sql = "SELECT ID_EJEMPLAR FROM EJEMPLARES
3         WHERE ID_LIBRO = $cod_libro
4         AND ID_EJEMPLAR NOT IN (SELECT ID_EJEMPLAR FROM PRESTAMOS WHERE FECHA_DEVOLUCION IS NULL)
5         LIMIT 1";
6 $res = $conexion->query($sql);
```

Este código realiza una consulta para encontrar un libro que no esté prestado en estos momentos.

De la siguiente query queremos el ID_EJEMPLAR de la tabla EJEMPLARES filtrando los ejemplares que solo pertenecen al libro solicitado.

NOT IN (SELECT ... WHERE FECHA_DEVOLUCION IS NULL): Esta subconsulta buscamos todos los ejemplares que están actualmente prestados indicando que la fecha de devolución está vacía o sea NULL. NOT IN los descarta, dejando solo los ejemplares que están libres.

\$conexion->query(\$sql): Ejecuta la consulta y guarda el resultado en \$res.

***Esta instrucción se asemeja a la siguiente select en bases de datos: ***

```
SELECT E.ID_EJEMPLAR FROM EJEMPLARES E LEFT JOIN PRESTAMOS P ON
E.ID_EJEMPLAR = P.ID_EJEMPLAR AND P.FECHA_DEVOLUCION IS NULL WHERE
E.ID_LIBRO = $cod_libro AND P.ID_EJEMPLAR IS NULL
```

TABLA PRESTAMOS: INSERT



```
1 $sql2 = "INSERT INTO PRESTAMOS (ID_USER, ID_EJEMPLAR, FECHA_ALQUILER, FECHA_DEVOLUCION)
2       VALUES ($id_user, " . $ej['ID_EJEMPLAR'] . ", CURDATE(), NULL)";
```

Este código registra un nuevo préstamo en la base de datos.

La sentencia SQL realiza una inserción de datos en la tabla PRESTAMOS con los siguientes valores:

Añadimos un nuevo registro en la tabla PRESTAMOS definiendo las columnas de la tabla en las que vamos a guardar la información sobre el nuevo préstamo.

\$id_user: Introduce el id del usuario que solicita el préstamo, tomado directamente de la variable de PHP.

" . \$ej['ID_EJEMPLAR'] . ": Introduce el id del ejemplar disponible que se va a entregar, concatenando el valor obtenido del array \$ej.

***Esta instrucción se asemeja a la siguiente select en bases de datos: ***

INSERT INTO PRESTAMOS VALUES (1045, 284, '2026-05-23', NULL);



TABLA LIBROS: INSERT

```
1 $sql = "INSERT INTO LIBROS (ISBN, TITULO, EDITORIAL, GENERO, NUM_COPIAS) VALUES ('$isbn', '$titulo', '$editorial', '$genero', $ejemplares)";
2
3 if ($conexion->query($sql)) {
4     $libro_id = $conexion->insert_id;
5
6     for ($i = 0; $i < $ejemplares; $i++) {
7         $conexion->query("INSERT INTO EJEMPLARES (ID_LIBRO) VALUES ($libro_id)");
8     }
9 }
```

Inserción del libro:

Preparamos la consulta para insertar un nuevo libro en la tabla LIBROS, guardando sus datos.

if (\$conexion->query(\$sql)): Ejecuta la query anterior y mediante una condición comprobamos si la insert del libro se ha realizado con éxito.

\$libro_id = \$conexion->insert_id; Recuperamos el identificador numérico (ID) que la base de datos le acaba de asignar a ese nuevo libro insertado.

for (\$i = 0; \$i < \$ejemplares; \$i++): Iniciamos un bucle repetitivo que se ejecutará tantas veces como el número de copias físicas tenga el libro.

INSERT INTO EJEMPLARES (ID_LIBRO) VALUES (\$libro_id): Ejecutamos la query de inserción por cada vuelta del bucle en la tabla EJEMPLARES.

***Esta instrucción se asemeja a la siguiente select en bases de datos: ***

INSERT INTO LIBROS VALUES ('978-3-16', 'Don Quijote', 'Planeta', 'Novela', 3);

TABLA EJEMPLARES: INSERT

```
1  for ($i = 0; $i < $cantidad; $i++) {  
2      $conexion->query("INSERT INTO EJEMPLARES (ID_LIBRO) VALUES ($libro_id)");  
3  }
```

INSERT INTO EJEMPLARES (ID_LIBRO) VALUES (\$libro_id): Especificamos que se va a añadir un registro en la tabla EJEMPLARES, insertando únicamente la columna que asocia el ejemplar con su libro correspondiente.

***Esta instruccion se asemeja a la siguiente select en bases de datos: ***

INSERT INTO EJEMPLARES (ID_LIBRO) VALUES (7), (7), (7);

TABLA EJEMPLARES: INSERT

```
1  $libros = $conexion->query("SELECT *,  
2      (SELECT COUNT(*) FROM EJEMPLARES E  
3      WHERE E.ID_LIBRO = LIBROS.ID_LIBRO  
4      AND E.ID_EJEMPLAR NOT IN (SELECT ID_EJEMPLAR FROM PRESTAMOS WHERE FECHA_DEVOLUCION IS NULL)  
5      ) AS DISPONIBLES  
6      FROM LIBROS ORDER BY DISPONIBLES DESC, TITULO ASC");
```

SELECT * FROM LIBROS ORDER BY DISPONIBLES DESC, TITULO ASC

Indicamos que queremos recuperar todas las columnas y datos generales de las obras, especificando que la query se realiza sobre la tabla LIBROS. Ordenamos el listado mostrando primero los libros con mayor cantidad de copias disponibles en descendente y por el titulo del libro en ascendente

Subconsulta de conteo de disponibles:

(SELECT COUNT(*) FROM EJEMPLARES E ...) WHERE E.ID_LIBRO = LIBROS.ID_LIBRO AND E.ID_EJEMPLAR NOT IN (...) (SELECT ID_EJEMPLAR FROM PRESTAMOS WHERE FECHA_DEVOLUCION IS NULL)



Ejecutamos una subconsulta que cuenta las copias físicas asociadas a cada libro, filtrando el conteo para asegurarnos de relacionar cada ejemplar físico únicamente con su correspondiente libro de la consulta principal. Aplicamos una condición de exclusión para no contar los libros que no se pueden prestar en este momento. Buscamos en el registro de alquileres todos los ejemplares que tiene un usuario actualmente.

***Esta instruccion se asemeja a la siguiente select en bases de datos: ***

```
SELECT L.*, COUNT(E.ID_EJEMPLAR) AS DISPONIBLES FROM LIBROS L JOIN EJEMPLARES  
E ON L.ID_LIBRO = E.ID_LIBRO AND E.ID_EJEMPLAR NOT IN (SELECT ID_EJEMPLAR FROM  
PRESTAMOS WHERE FECHA_DEVOLUCION IS NULL) GROUPBY L.ID_LIBRO ORDER BY  
DISPONIBLES DESC, TITULO ASC;
```

TABLAS PRESTAMOS, EJEMPLARES, LIBROS: SELECT + JOIN

```
1 $prestamos = $conexion->query("SELECT p.ID_PRESTAMO, l.TITULO, p.FECHA_ALQUILER, p.FECHA_DEVOLUCION  
2 FROM PRESTAMOS p  
3 JOIN EJEMPLARES e ON p.ID_EJEMPLAR = e.ID_EJEMPLAR  
4 JOIN LIBROS l ON e.ID_LIBRO = l.ID_LIBRO  
5 WHERE p.ID_USER = $id_user AND p.FECHA_DEVOLUCION IS NULL  
6 ORDER BY p.FECHA_DEVOLUCION ASC");  
7  
8 $libros_select = $conexion->query("SELECT * FROM LIBROS ORDER BY TITULO");
```

```
SELECT p.ID_PRESTAMO, l.TITULO, ... FROM PRESTAMOS p JOIN EJEMPLARES e ON  
p.ID_EJEMPLAR = e.ID_EJEMPLAR JOIN LIBROS l ON e.ID_LIBRO = l.ID_LIBRO WHERE  
p.ID_USER = $id_user AND p.FECHA_DEVOLUCION IS NULL ORDER BY  
p.FECHA_DEVOLUCION ASC
```

Especificamos las columnas exactas que queremos obtener. Definimos la tabla PRESTAMOS como tabla principal de la consulta y asignamos el alias p para simplificar el código. Mediante la primera join, conectamos la tabla de PRESTAMOS (p) con EJEMPLARES (e) relacionándolas a través del identificador del ejemplar.

La segunda join, juntamos la tabla anterior (EJEMPLARES e) con LIBROS (l) mediante el código del libro para poder acceder a los datos. Filtramos los registros para mostrar únicamente los libros prestados al usuario que todavía no han sido devueltos (IS NULL).

***Esta instruccion se asemeja a la siguiente select en bases de datos: ***

```
SELECT p.ID_PRESTAMO, l.TITULO, ... FROM PRESTAMOS p JOIN EJEMPLARES e ON  
p.ID_EJEMPLAR = e.ID_EJEMPLAR JOIN LIBROS l ON e.ID_LIBRO = l.ID_LIBRO WHERE
```



```
p.ID_USER = $id_user AND p.FECHA_DEVOLUCION IS NULL ORDER BY  
p.FECHA_DEVOLUCION ASC
```

TABLA USUARIOS: SELECT

```
1 $sql = "SELECT * FROM USUARIOS WHERE NOMBRE_USER='$usuario_login' OR DNI_USER='$usuario_login';"  
2
```

Indicamos a la base de datos que busque toda la información disponible en la tabla USUARIOS. Filtramos la búsqueda para encontrar únicamente la cuenta del usuario que está intentando acceder al sistema. Hacemos una comprobación de si el texto introducido en el formulario coincide exactamente con el nombre de usuario registrado en la base de datos permitiendo que la búsqueda también sea válida si el texto introducido coincide con el campo del DNI.

***Esta instrucción se asemeja a la siguiente select en bases de datos: ***

```
SELECT * FROM USUARIOS WHERE NOMBRE_USER = '$usuario_login' UNION  
SELECT * FROM USUARIOS WHERE DNI_USER = '$usuario_login';
```

TABLA USUARIOS: SELECT

```
1 $check = $conexion->query("SELECT * FROM usuarios WHERE usuario='$usuario' OR email='$email' OR DNI_usu='$dni');"  
2
```

Solicitamos recuperar todos los datos de las filas que cumplan con los filtros en la tabla usuarios, comprobando si ya existe un registro con el mismo nombre de usuario que se intenta registrar. Verificamos si el correo electrónico introducido ya se encuentra registrado en el sistema y comprobamos si el DNI pertenece a otra cuenta.

***Esta instrucción se asemeja a la siguiente select en bases de datos: ***



```
SELECT * FROM usuarios WHERE usuario='$usuario' OR email='$email' OR  
DNI_usu='$dni';
```

4.- BASE DE DATOS CON LA ASIGNATURA DE PROGRAMACION

En este apartado se va a explicar que código de programación afecta a la base de datos (conexión, consultas, modificaciones, inserciones, y borrados).

Para ello se ha desarrollado la aplicación orientada a objetos por completo la cual consta tanto de una conexión a una base de datos como un menú interactivo donde se muestran las acciones a realizar con la base de datos.

Mostramos a continuación, el código de la conexión a la base de datos explicado anteriormente en la asignatura de programación y seguidamente se explica las operaciones de consultas, modificaciones, inserciones, y borrados tomando como ejemplo la tabla LIBROS.

También se explica la instrucción sql que se asemeja a dicho código.

CODIGO DE CONEXION A LA BASE DE DATOS:

```
1  public class testConexion {  
2  
3      public static void main(String[] args) {  
4  
5          String hostname = "localhost";  
6          String database = "biblioteca";  
7          String username = "root";  
8          String password = "";  
9          Connection cnx = null;  
10  
11         String url = "jdbc:mysql://" + hostname + ":3306/" + database + "?useSSL=false";  
12  
13         cnx = InputOutput.getConnection(url, username, password);  
14         if (cnx == null) {  
15  
16             System.out.println("Sin conexión a la base de datos :(");  
17             return;  
18         }  
19  
20         System.out.println("Conexión a la base de datos exitosa !!!");  
21     }  
22 }
```

```
1 public class InputOutput {
2
3     public static Connection getConexion(String u, String n, String p) {
4
5         try {
6             Connection conn = null;
7             conn = DriverManager.getConnection(u, n, p);
8             return conn;
9         } catch (SQLException e) {
10             e.printStackTrace();
11             return null;
12         }
13     }
14 }
15
```

TOMANDO COMO EJEMPLO LA TABLA LIBROS OPERACIONES DE CONSULTAS, MODIFICACIONES, INSERCCIONES, Y BORRADOS

Consultar Libro (SELECT):

```
1 String sql = "SELECT id_libro, titulo, isbn, editorial, genero, num_copias FROM libros";
2
```

En esta consulta SQL guardamos mediante una variable llamada sql la cual obtiene datos de la tabla libros los cuales los seleccionamos para mostrarlos.

La instrucción de SQL que se asemeja a dicho código es:

SELECT id_libro, titulo, isbn, editorial, genero, num_copias FROM libros;



Insertar Libro (INSERT):

```
1 String sql = "INSERT INTO libros (id_libro, titulo, isbn, editorial, genero, num_copias) VALUES (?, ?, ?, ?, ?, ?)";  
2
```

Este código muestra una sentencia SQL para insertar datos mediante una variable llamada `sql`. Los signos de interrogación se conocen como parámetros de sustitución y sirven para indicar a la base de datos cuáles son los nuevos valores que se van a introducir en la base de datos.

Modificar Libro (UPDATE) :

```
1 String sql = "UPDATE libros SET editorial = ? WHERE id_libro = ?";  
2
```

Esta instrucción sirve para actualizar o modificar datos que ya existen en la base de datos. El cambio que se va a realizar mediante el `SET` indicamos qué columna queremos cambiar y qué nuevo valor vamos a introducir. La primera interrogación es para introducir la nueva editorial donde el segundo interrogante es el ID del libro que queremos modificar.

La instrucción de SQL que se asemeja a dicho código es:

```
UPDATE libros SET editorial = 'Planeta' WHERE id_libro = 5;
```



Borrar Libro (DELETE) :

```
1 String sql = "DELETE FROM libros WHERE id_libro = ?";  
2
```

La instrucción SQL sirve para eliminar un registro específico de la base de datos.

La instrucción de SQL que se asemeja a dicho código es:

DELETE FROM libros WHERE id_libro = 5;

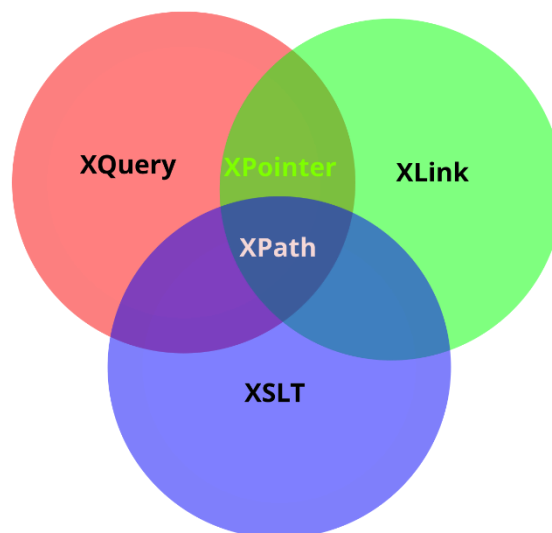
Lenguaje de marcas

En nuestra biblioteca hemos comenzado a modernizar la forma en la que gestionamos y estructuramos la información, y para ello hemos incorporado distintas herramientas y conceptos trabajados en la asignatura de Lenguajes de Marcas.

Durante este proceso hemos creado y validado documentos XML utilizando DTD y XML Schema, definiendo estructuras completas con elementos, atributos, restricciones y patrones que garantizan que los datos sean consistentes y fáciles de mantener.

También hemos aplicado consultas XPath y XQuery para localizar, filtrar y procesar información de manera precisa, algo especialmente útil para manejar catálogos, préstamos o registros internos.

Además, hemos desarrollado transformaciones XSLT que nos permiten presentar los datos en distintos formatos, como HTML, XML o PDF, facilitando su visualización y reutilización.



1. Transformación de tablas SQL a XML

Hemos transformado las tablas SQL del proyecto a documentos XML estructurados, definiendo para cada una su correspondiente DTD. Este proceso nos permite representar los datos de forma jerárquica, controlar su estructura mediante validación y preparar la información para futuras consultas y transformaciones.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE biblioteca SYSTEM "file:/C:/Users/ainur/Downloads/
:biblioteca">
```

```
<!--===== LIBROS =====-->
```

```
<libros>
  <libro id_libro="1">
    <titulo>El Quijote</titulo>
    <isbn>9788491050295</isbn>
    <editorial>Alfaguara</editorial>
    <genero>Clásico</genero>
    <num_copias>4</num_copias>
  </libro>

  <libro id_libro="2">
    <titulo>1984</titulo>
    <isbn>9780451524935</isbn>
    <editorial>Penguin</editorial>
    <genero>Distopía</genero>
    <num_copias>6</num_copias>
  </libro>

  <libro id_libro="3">
    <titulo>Fundación</titulo>
    <isbn>9780553293357</isbn>
    <editorial>Gnome Press</editorial>
    <genero>Ciencia Ficción</genero>
    <num_copias>5</num_copias>
  </libro>
</libros>
```

```
<!--===== USUARIOS =====-->
```

```
<usuarios>
  <usuario id_user="1">
    <dni_user>12345678A</dni_user>
    <nombre_user>Ainur</nombre_user>
    <telefono_user>666555444</telefono_user>
    <password_user>pass123</password_user>
    <nss>123456789012</nss>
    <id_penalizacion>1</id_penalizacion>
  </usuario>

  <usuario id_user="2">
    <dni_user>87654321B</dni_user>
    <nombre_user>Markel</nombre_user>
    <telefono_user>600111222</telefono_user>
    <password_user>markel22</password_user>
    <nss>987654321000</nss>
    <id_penalizacion>2</id_penalizacion>
  </usuario>

  <usuario id_user="3">
    <dni_user>11223344C</dni_user>
    <nombre_user>Enok</nombre_user>
    <telefono_user>699888777</telefono_user>
    <password_user>enokD</password_user>
    <nss>111222333444</nss>
    <id_penalizacion>3</id_penalizacion>
  </usuario>
</usuarios>
```

```
<!--===== AUTORES =====-->
```

```
<autores>
  <autor id_autor="1">
    <nombre>Miguel</nombre>
    <apellidos>de Cervantes</apellidos>
    <nacionalidad>España</nacionalidad>
  </autor>

  <autor id_autor="2">
    <nombre>George</nombre>
    <apellidos>Orwell</apellidos>
    <nacionalidad>Reino Unido</nacionalidad>
  </autor>

  <autor id_autor="3">
    <nombre>Isaac</nombre>
    <apellidos>Asimov</apellidos>
    <nacionalidad>Rusia</nacionalidad>
  </autor>
</autores>
```

```
<!--===== RELACIÓN LIBROS--AUTORES =====-->
```

```
<libros_autores>
  <relacion id_libro="1" id_autor="1"/>
  <relacion id_libro="2" id_autor="2"/>
  <relacion id_libro="3" id_autor="3"/>
</libros_autores>
```

```
<!--===== EJEMPLARES =====-->
```

```
<ejemplares>
  <ejemplar id_ejemplar="1">
    <estado_fisico>Bien</estado_fisico>
    <id_libro>1</id_libro>
  </ejemplar>

  <ejemplar id_ejemplar="2">
    <estado_fisico>Regular</estado_fisico>
    <id_libro>2</id_libro>
  </ejemplar>

  <ejemplar id_ejemplar="3">
    <estado_fisico>Excelente</estado_fisico>
    <id_libro>3</id_libro>
  </ejemplar>

  <ejemplar id_ejemplar="4">
    <estado_fisico>Deteriorado</estado_fisico>
    <id_libro>1</id_libro>
  </ejemplar>
</ejemplares>
```

```
<!--===== PENALIZACIONES =====-->
```

```
<penalizaciones>
  <penalizacion id_penalizacion="1">
    <descripcion>Retraso leve</descripcion>
    <num_dias>3</num_dias>
  </penalizacion>

  <penalizacion id_penalizacion="2">
    <descripcion>Retraso grave</descripcion>
    <num_dias>15</num_dias>
  </penalizacion>

  <penalizacion id_penalizacion="3">
    <descripcion>Pérdida de ejemplar</descripcion>
    <num_dias>30</num_dias>
  </penalizacion>
</penalizaciones>
```

```
<!--===== PRESTAMOS =====-->
```

```
<prestamos>
  <prestamo id_prestamo="1">
    <fecha_alquiler>2024-05-01</fecha_alquiler>
    <fecha_devolucion>2024-05-10</fecha_devolucion>
    <id_ejemplar>1</id_ejemplar>
    <id_user>1</id_user>
  </prestamo>

  <prestamo id_prestamo="2">
    <fecha_alquiler>2024-05-03</fecha_alquiler>
    <fecha_devolucion>2024-05-18</fecha_devolucion>
    <id_ejemplar>2</id_ejemplar>
    <id_user>2</id_user>
  </prestamo>

  <!-- Préstamo aún no devuelto -->
  <prestamo id_prestamo="3">
    <fecha_alquiler>2024-05-20</fecha_alquiler>
    <fecha_devolucion></fecha_devolucion>
    <id_ejemplar>3</id_ejemplar>
    <id_user>3</id_user>
  </prestamo>
</prestamos>
```

```
/:biblioteca
```

Información



BibliotecaDukes.xml es bien-formado

Información



BibliotecaDukes.xml is valid

Su respectiva DTD

```

1 <!ELEMENT biblioteca(libros,autores,libros_autores,penalizaciones,usuarios,ejemplares,prestamos)>
2
3 <!--===== LIBROS =====>
4 <ELEMENT libros(libro+)>
5
6 <ELEMENT libro(titulo,isbn,editorial,genero,num_copias)>
7 <!ATTLIST libro
8 | id_libro ID #REQUIRED>
9
10 <ELEMENT titulo(#PCDATA)>
11 <ELEMENT isbn(#PCDATA)>
12 <ELEMENT editorial(#PCDATA)>
13 <ELEMENT genero(#PCDATA)>
14 <ELEMENT num_copias(#PCDATA)>
15
16
17 <!--===== AUTORES =====>
18 <ELEMENT autores(autor+)>
19
20 <ELEMENT autor(nombre,apellidos,nacionalidad)>
21 <!ATTLIST autor
22 | id_autor ID #REQUIRED>
23
24 <ELEMENT nombre(#PCDATA)>
25 <ELEMENT apellidos(#PCDATA)>
26 <ELEMENT nacionalidad(#PCDATA)>
27
28
29 <!--===== RELACIÓN LIBROS ~AUTORES =====>
30 <ELEMENT libros_autores(relacion+)>
31
32 <ELEMENT relacion EMPTY>
33 <!ATTLIST relacion
34 | id_libro IDREF #REQUIRED
35 | id_autor IDREF #REQUIRED>
36
37
38 <!--===== PENALIZACIONES =====>
39 <ELEMENT penalizaciones(penalizacion+)>
40
41 <ELEMENT penalizacion(descripcion,num_dias)>
42 <!ATTLIST penalizacion
43 | id_penalizacion ID #REQUIRED>
44
45 <ELEMENT descripcion(#PCDATA)>
46 <ELEMENT num_dias(#PCDATA)>
47
48
49 <!--===== USUARIOS =====>
50 <ELEMENT usuarios(usuario+)>
51
52 <ELEMENT usuario(dni_user,nombre_user,telefono_user,password_user,nss,id_penalizacion)>
53 <!ATTLIST usuario
54 | id_user ID #REQUIRED>
55
56 <ELEMENT dni_user(#PCDATA)>
57 <ELEMENT nombre_user(#PCDATA)>
58 <ELEMENT telefono_user(#PCDATA)>
59 <ELEMENT password_user(#PCDATA)>
60 <ELEMENT nss(#PCDATA)>
61 <ELEMENT id_penalizacion(#PCDATA)>
62
63
64 <!--===== EJEMPLARES =====>
65 <ELEMENT ejemplares(ejemplar+)>
66
67 <ELEMENT ejemplar(estado_fisico,id_libro)>
68 <!ATTLIST ejemplar
69 | id_ejemplar ID #REQUIRED>
70
71 <ELEMENT estado_fisico(#PCDATA)>
72 <ELEMENT id_libro(#PCDATA)>
73
74
75 <!--===== PRESTAMOS =====>
76 <ELEMENT prestamos(prestamo+)>
77
78 <ELEMENT prestamo(fecha_alquiler,fecha_devolucion,id_ejemplar,id_user)>
79 <!ATTLIST prestamo
80 | id_prestamo ID #REQUIRED>
81
82 <ELEMENT fecha_alquiler(#PCDATA)>
83 <ELEMENT fecha_devolucion(#PCDATA)>
84 <ELEMENT id_ejemplar(#PCDATA)>
85 <ELEMENT id_user(#PCDATA)>

```

Su respectivo W3C Schema

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
3   <xs:element name="biblioteca">
4     <xs:complexType>
5       <xs:sequence>
6         <xs:element ref="libros"/>
7         <xs:element ref="autores"/>
8         <xs:element ref="libros_autores"/>
9         <xs:element ref="penalizaciones"/>
10        <xs:element ref="usuarios"/>
11        <xs:element ref="ejemplares"/>
12        <xs:element ref="prestamos"/>
13      </xs:sequence>
14    </xs:complexType>
15  </xs:element>
16
17  <xs:element name="libros">
18    <xs:complexType>
19      <xs:sequence>
20        <xs:element name="libro" maxOccurs="unbounded"/>
21      </xs:sequence>
22    </xs:complexType>
23  </xs:element>
24
25  <xs:element name="libro">
26    <xs:complexType>
27      <xs:sequence>
28        <xs:element name="titulo" type="xs:string"/>
29        <xs:element name="isbn" type="xs:string"/>
30        <xs:element name="editorial" type="xs:string"/>
31        <xs:element name="genero" type="xs:string"/>
32        <xs:element name="num_copias" type="xs:string"/>
33      </xs:sequence>
34      <xs:attribute name="id_libro" type="xs:string" use="required"/>
35    </xs:complexType>
36  </xs:element>
37
38  <xs:element name="autores">
39    <xs:complexType>
40      <xs:sequence>
41        <xs:element ref="autor" maxOccurs="unbounded"/>
42      </xs:sequence>
43    </xs:complexType>
44  </xs:element>
45
46  <xs:element name="autor">
47    <xs:complexType>
48      <xs:sequence>
49        <xs:element name="nombre" type="xs:string"/>
50        <xs:element name="apellidos" type="xs:string"/>
51        <xs:element name="nacionalidad" type="xs:string"/>
52      </xs:sequence>
53      <xs:attribute name="id_autor" type="xs:string" use="required"/>
54    </xs:complexType>
55  </xs:element>
56
57  <xs:element name="libros_autores">
58    <xs:complexType>
59      <xs:sequence>
60        <xs:element ref="relacion" maxOccurs="unbounded"/>
61      </xs:sequence>
62    </xs:complexType>
63  </xs:element>
64
65  <xs:element name="relacion">
66    <xs:complexType>
67      <xs:sequence>
68        <xs:element name="descripcion" type="xs:string"/>
69        <xs:element name="num_dias" type="xs:string"/>
70      </xs:sequence>
71      <xs:attribute name="id_autor" type="xs:string" use="required"/>
72      <xs:attribute name="id_libro" type="xs:string" use="required"/>
73    </xs:complexType>
74  </xs:element>
75
76  <xs:element name="penalizaciones">
77    <xs:complexType>
78      <xs:sequence>
79        <xs:element ref="penalizacion" maxOccurs="unbounded"/>
80      </xs:sequence>
81    </xs:complexType>
82  </xs:element>
83
84  <xs:element name="penalizacion">
85    <xs:complexType>
86      <xs:sequence>
87        <xs:element name="descripcion" type="xs:string"/>
88        <xs:element name="num_dias" type="xs:string"/>
89      </xs:sequence>
90      <xs:attribute name="id_penalizacion" type="xs:string" use="required"/>
91    </xs:complexType>
92  </xs:element>
93
94  <xs:element name="usuarios">
95    <xs:complexType>
96      <xs:sequence>
97        <xs:element ref="usuario" maxOccurs="unbounded"/>
98      </xs:sequence>
99    </xs:complexType>
100  </xs:element>
101
102  <xs:element name="usuario">
103    <xs:complexType>
104      <xs:sequence>
105        <xs:element name="dni_user" type="xs:string"/>
106        <xs:element name="nombre_user" type="xs:string"/>
107        <xs:element name="telefono_user" type="xs:string"/>
108        <xs:element name="password_user" type="xs:string"/>
109        <xs:element name="nss" type="xs:string"/>
110        <xs:element name="id_penalizacion" type="xs:string"/>
111      </xs:sequence>
112      <xs:attribute name="id_user" type="xs:string" use="required"/>
113    </xs:complexType>
114  </xs:element>
115
116  <xs:element name="ejemplares">
117    <xs:complexType>
118      <xs:sequence>
119        <xs:element ref="ejemplar" maxOccurs="unbounded"/>
120      </xs:sequence>
121    </xs:complexType>
122  </xs:element>
123
124  <xs:element name="ejemplar">
125    <xs:complexType>
126      <xs:sequence>
127        <xs:element name="estado_fisico" type="xs:string"/>
128        <xs:element name="id_libro" type="xs:string"/>
129      </xs:sequence>
130      <xs:attribute name="id_ejemplar" type="xs:string" use="required"/>
131    </xs:complexType>
132  </xs:element>
133
134  <xs:element name="prestamos">
135    <xs:complexType>
136      <xs:sequence>
137        <xs:element ref="prestamo" maxOccurs="unbounded"/>
138      </xs:sequence>
139    </xs:complexType>
140  </xs:element>
141
142  <xs:element name="prestamo">
143    <xs:complexType>
144      <xs:sequence>
145        <xs:element name="fecha_alquiler" type="xs:string"/>
146        <xs:element name="fecha_devolucion" type="xs:string"/>
147        <xs:element name="id_ejemplar" type="xs:string"/>
148        <xs:element name="id_user" type="xs:string"/>
149      </xs:sequence>
150      <xs:attribute name="id_prestamo" type="xs:string" use="required"/>
151    </xs:complexType>
152  </xs:element>
153
154  </xs:schema>

```

2. XSLT

XSLT (*Extensible Stylesheet Language Transformations*) es un lenguaje diseñado para **transformar documentos XML** en otros formatos, como **HTML**, **texto plano** o **incluso otros XML**. Su función principal es permitir que los datos estructurados en XML puedan mostrarse de forma visual, ordenada y adaptada a una página web.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:output method="html" indent="yes" encoding="UTF-8"/>

  <!--=====
  ||| TEMPLATE PRINCIPAL
  =====-->
  <xsl:template match="/" biblioteca">
    <html>
    <head>
      <title>Biblioteca | Informe General</title>
      <style>
        body { font-family: Arial; margin: 20px; background: #f4f4f4; }
        h1 { color: #333; }
        .bloque { background: white; padding: 20px; margin-bottom: 25px; border-radius: 10px; }
        table { width: 100%; border-collapse: collapse; margin-top: 10px; }
        th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
        th { background: #333; color: white; }
      </style>
    </head>
    <body>
      <h1>Biblioteca | Informe General</h1>

      <!-- BLOQUE LIBROS -->
      <div class="bloque">
        <h2>Libros</h2>
        <table>
          <tr>
            <th>ID</th>
            <th>Título</th>
            <th>ISBN</th>
            <th>Editorial</th>
            <th>Género</th>
            <th>Copias</th>
          </tr>
          <tr>
            <td data-cs="6" data-kind="parent">Aquí se insertan las filas-->
            <td data-kind="ghost"><xsl:apply-templates select="libros/libro"/>
            </td>
          </tr>
        </table>
      </div>

      <!-- BLOQUE AUTORES -->
      <div class="bloque">
        <h2>Autores</h2>
        <table>
          <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Apellidos</th>
            <th>Nacionalidad</th>
          </tr>
          <tr>
            <td data-cs="4" data-kind="parent">Aquí se insertan las filas-->
            <td data-kind="ghost"><xsl:apply-templates select="autores/autor"/>
            </td>
          </tr>
        </table>
      </div>

      <!-- BLOQUE PRÉSTAMOS -->
      <div class="bloque">
        <h2>Préstamos</h2>
        <table>
          <tr>
            <th>ID</th>
            <th>Fecha alquiler</th>
            <th>Fecha devolución</th>
            <th>ID Ejemplar</th>
            <th>ID Usuario</th>
          </tr>
          <tr>
            <td data-cs="5" data-kind="parent">Aquí se insertan las filas-->
            <td data-kind="ghost"><xsl:apply-templates select="prestamos/prestamo"/>
            </td>
          </tr>
        </table>
      </div>
    </body>
  </html>
</xsl:template>

  <!--=====
  ||| FILAS DE LIBROS
  =====-->
  <xsl:template match="libro">
    <tr>
      <td><xsl:value-of select="@id_libro"/></td>
      <td><xsl:value-of select="titulo"/></td>
      <td><xsl:value-of select="isbn"/></td>
      <td><xsl:value-of select="editorial"/></td>
      <td><xsl:value-of select="genero"/></td>
      <td><xsl:value-of select="num_copias"/></td>
    </tr>
  </xsl:template>

  <!--=====
  ||| FILAS DE AUTORES
  =====-->
  <xsl:template match="autor">
    <tr>
      <td><xsl:value-of select="@id_autor"/></td>
      <td><xsl:value-of select="nombre"/></td>
      <td><xsl:value-of select="apellidos"/></td>
      <td><xsl:value-of select="nacionalidad"/></td>
    </tr>
  </xsl:template>

  <!--=====
  ||| FILAS DE PRÉSTAMOS
  =====-->
  <xsl:template match="prestamo">
    <tr>
      <td><xsl:value-of select="@id_prestamo"/></td>
      <td><xsl:value-of select="fecha_alquiler"/></td>
      <td><xsl:choose>
        <xsl:when test="string-length(fecha_devolucion) > 0">
          <xsl:value-of select="fecha_devolucion"/>
        </xsl:when>
        <xsl:otherwise>
          <span style="color:red;">Pendiente</span>
        </xsl:otherwise>
      </xsl:choose>
      <td><xsl:value-of select="id_ejemplar"/></td>
      <td><xsl:value-of select="id_usuario"/></td>
    </tr>
  </xsl:template>
```

Parte 1 → Plantilla principal (match="/biblioteca")

Es la que genera **toda la página HTML**:

- <html>
- <head> con estilos CSS
- <body> con los bloques de:
 - Libros
 - Autores



- Usuarios
- Préstamos

Dentro de cada bloque hay una tabla con sus cabeceras.

Parte 2 → Plantillas específicas para cada nodo

```
<xsl:template match="libro"> ... </xsl:template>
```

genera **una fila** <tr> con los datos del nodo.

Cómo funciona cada bloque

Ejemplo: BLOQUE LIBROS

En la plantilla principal:

```
<xsl:apply-templates select="libros/libro"/>
```

Esto significa:

“Por cada <libro> dentro de <libros>, aplica la plantilla que coincida con match="libro"”.

La plantilla de libro:

```
<xsl:template match="libro">

  <tr>

    <td><xsl:value-of select="@id_libro"/></td>

    <td><xsl:value-of select="titulo"/></td>

    <td><xsl:value-of select="isbn"/></td>

    <td><xsl:value-of select="editorial"/></td>

    <td><xsl:value-of select="genero"/></td>

    <td><xsl:value-of select="num_copias"/></td>

  </tr>

</xsl:template>
```

Genera **una fila completa** con todos los datos del libro.

3. Consultas XPATH sencillas

- Obtener todos los títulos de los libros

Evaluar XPath

XPath:

`/biblioteca/libros/libro/titulo`

Aceptar Cancel

Resultado

```
1 <titulo>El Quijote</titulo>
2 <titulo>1984</titulo>
3 <titulo>Fundación</titulo>
4
```

- Obtener los usuarios registrados

Evaluar XPath

XPath:

`/biblioteca/usuarios/usuario/nombre_user`

Aceptar Cancel

Resultado

```
1 <nombre_user>Ainur</nombre_user>
2 <nombre_user>Markel</nombre_user>
3 <nombre_user>Enok</nombre_user>
4
```

- Obtener las penalizaciones disponibles

Evaluar XPath ✕

XPath:

Resultado

```
1 <descripcion>Retraso leve</descripcion>  
2 <descripcion>Retraso grave</descripcion>  
3 <descripcion>Pérdida de ejemplar</descripcion>  
4
```

4. Consultas anidadas

- Obtener el título del libro cuyo autor es “George Orwell”

Evaluar XPath

XPath:

`/biblioteca/libros/libro[@id_libro = /biblioteca/libros_autores/relacion[@id_autor='2']/@id_libro]/titulo`

Aceptar Cancel

Resultado

1	<code><titulo>1984</titulo></code>
2	

- Obtener los títulos de los libros que tienen ejemplares deteriorados

Evaluar XPath

XPath:

`/biblioteca/libros/libro[@id_libro = /biblioteca/ejemplares/ejemplar[estado_fisico='Deteriorado']/@id_libro]/titulo`

Aceptar Cancel

Resultado

1	<code><titulo>El Quijote</titulo></code>
2	

- **Obtener el nombre del autor del libro “Fundación”**

Evaluar XPath

XPath:

`/biblioteca/autores/autor[@id_autor = /biblioteca/libros_autores/relacion[@id_libro='3']/@id_autor]/nombre`

Aceptar Cancel

Resultado

```
1 <nombre>|saac</nombre>
2
```

- **Obtener los títulos de los libros que tienen ejemplares deteriorados**

Evaluar XPath

XPath:

`/biblioteca/usuarios/usuario[@id_user = /biblioteca/prestamos/prestamo[fecha_devolucion=""]/id_user]/nombre_user`

Aceptar Cancel

Resultado

```
1 <nombre_user>Enol</nombre_user>
2
```



5. Consultas con funciones

- **Obtener los usuarios que han alquilado libros de autores españoles**

```
/biblioteca/usuarios/usuario[
  @id_user = /biblioteca/prestamos/prestamo[
    id_ejemplar = /biblioteca/ejemplares/ejemplar[
      id_libro = /biblioteca/libros/libro[
        @id_libro = /biblioteca/libros_autores/relacion[
          @id_autor = /biblioteca/autores/autor[nacionalidad='España']/@id_autor
        ]/@id_libro
      ]/@id_libro
    ]/@id_ejemplar
  ]/id_user
]/nombre_user
```

Resultado

```
1 <nombre_user>Ainur</nombre_user>
2
```

- **Libros cuyo título tiene más de 6 caracteres**

```
/biblioteca/libros/libro[string-length(titulo) > 6]/titulo
```

Resultado

```
1 <titulo>El Quijote</titulo>
2 <titulo>Fundación</titulo>
```



- Usuarios cuyo nombre empieza por la letra “M”

```
/biblioteca/usuarios/usuario[starts-with(nombre_user, 'M')]/nombre_user
```

Resultado

```
1 <nombre_user>Markel</nombre_user>  
2
```

- Libros cuyo género contiene la palabra “ción”

```
/biblioteca/libros/libro[contains(genero, 'ción')]/titulo
```

Resultado

```
1 <titulo>Fundación</titulo>  
2
```

- ISBN que terminan en “95”

```
/biblioteca/libros/libro[ends-with(isbn, '95')]/isbn
```

Resultado

- Usuarios cuyo teléfono tiene 9 dígitos

```
/biblioteca/usuarios/usuario[string-length(telefono_user)=9]/nombre_user
```

Resultado

```
1 <nombre_user>Ainur</nombre_user>  
2 <nombre_user>Markel</nombre_user>  
3 <nombre_user>Enol</nombre_user>
```



6. Consultas anidadas + funciones

- Obtener los usuarios que han alquilado libros cuyo título contiene la letra “a”

```
/biblioteca/usuarios/usuario[  
  @id_user = /biblioteca/prestamos/prestamo[  
    id_ejemplar = /biblioteca/ejemplares/ejemplar[  
      id_libro = /biblioteca/libros/libro[contains(titulo, 'a')]/@id_libro  
    ]/@id_ejemplar  
  ]/@id_user  
]/nombre_user
```

Resultado

```
1 <nombre_user>Enol</nombre_user>  
2
```



- **Obtener los autores cuyos libros tienen más de 5 copias**

```
/biblioteca/autores/autor[
  @id_autor = /biblioteca/libros_autores/relacion[
    @id_libro = /biblioteca/libros/libro[num_copias > 5]/@id_libro
  ]/@id_autor
]/nombre
```

Resultado

```
1 <nombre>George</nombre>
2
```

- **Obtener los usuarios que han alquilado libros de autores cuyo apellido tiene más de 6 letras**

```
/biblioteca/usuarios/usuario[
  @id_user = /biblioteca/prestamos/prestamo[
    id_ejemplar = /biblioteca/ejemplares/ejemplar[
      id_libro = /biblioteca/libros/libro[
        @id_libro = /biblioteca/libros_autores/relacion[
          @id_autor = /biblioteca/autores/autor[string-length(apellidos) > 6]/@id_autor
        ]/@id_libro
      ]/@id_libro
    ]/@id_ejemplar
  ]/id_user
]/nombre_user
```

Resultado

```
1 <nombre_user>Ainur</nombre_user>  
2
```

7. Consultas XQUERY explicación

En esta sección hemos decidido incorporar consultas **XQuery** para complementar el trabajo realizado con XML, DTD y XPath. Aunque su uso no era obligatorio, consideramos que aporta un valor añadido importante, ya que permite realizar consultas avanzadas, transformaciones y generación dinámica de datos a partir de documentos XML.

¿Qué es XQuery?

XQuery (XML Query Language) es un lenguaje diseñado específicamente para:

- Consultar documentos XML
- Filtrar, ordenar y transformar información estructurada
- Realizar operaciones complejas entre nodos
- Generar nuevos documentos XML o incluso HTML
- Integrar datos procedentes de múltiples fuentes XML

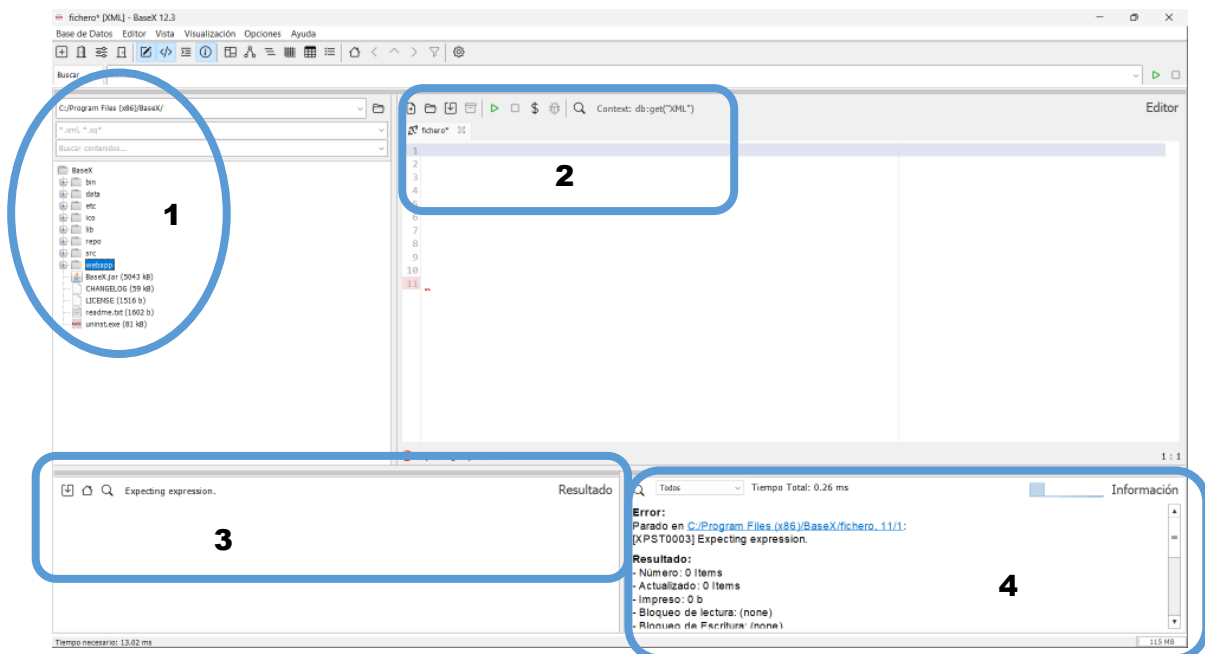
Es un estándar del W3C y se considera el equivalente a SQL dentro del ecosistema XML. Mientras que XPath permite seleccionar nodos, **XQuery amplía sus capacidades** añadiendo:

- Variables (let)
- Iteraciones (for)
- Condiciones (where)
- Construcción de nuevos elementos (return)
- Funciones avanzadas (matemáticas, de texto, de secuencias...)
- Joins entre diferentes partes del documento

Aplicación usada: **BASE**

BaseX es un sistema gestor de bases de datos nativo para XML y una de las herramientas más completas y potentes para trabajar con tecnologías del ecosistema XML. Desarrollado inicialmente por la Universidad de Constanza (Alemania), BaseX se ha convertido en un estándar académico y profesional para ejecutar **XQuery**, **XPath**, **XSLT**, y para gestionar grandes volúmenes de datos estructurados en XML.

Como funciona



1. Explorador de archivos (panel izquierdo)

Este panel muestra las carpetas y archivos del sistema. Sirve para:

- Navegar por tu PC desde BaseX.
- Abrir archivos XML, XQuery o bases de datos.
- Ver la estructura de directorios donde tienes tus proyectos.

No muestra bases de datos internas, solo archivos del sistema.



2. Editor de XQuery (panel central)

Es el área donde escribes y editas tus consultas XQuery.

Aquí puedes:

- Escribir consultas XQuery completas.
- Ejecutarlas con el botón ►.
- Ver errores marcados en rojo (como el que aparece en tu captura).
- Guardar tus scripts .xq.

Es el equivalente al “editor de código” de cualquier IDE.

3. Panel de resultados (parte inferior)

Aquí aparece **lo que devuelve tu consulta**.

Muestra:

- Resultado XML, texto o números.
- Errores de sintaxis (como *“Expecting expression”*).
- Información de ejecución:
 - Número de ítems devueltos
 - Bytes impresos
 - Elementos actualizados
 - Bloqueos de lectura/escritura

4. Panel de Resultados / Mensajes de Ejecución

Este panel aparece siempre que ejecutas una consulta XQuery. Su función es mostrar:

1. El error exacto que ha ocurrido
2. La ubicación del error
3. El resumen de ejecución
 - Número: 0 Items → No devolvió resultados
 - Actualizado: 0 Items → No se modificó nada
 - Impreso: 0 b → No se generó salida
 - Bloqueo de lectura: (none)
 - Bloqueo de escritura: (none)

8. Consultas XQUERY realizadas

- Listar los títulos de todos los libros

```
Context: db:get("XML")
fichero*
1 for $l in /biblioteca/libros/libro
2 return $l/titulo
3
```

Resultado

3 Resultados, 79 b

```
<titulo>El Quijote</titulo>
<titulo>1984</titulo>
<titulo>Fundación</titulo>
```

Explicación

for \$l in /biblioteca/libros/libro

- Recorre todos los nodos <libro> dentro de /biblioteca/libros.
- Cada libro se guarda en la variable \$l.

return \$l/titulo

- Para cada \$l, devuelve su hijo <titulo>.
- El resultado es una lista de nodos <titulo>:
 - El Quijote
 - 1984
 - Fundación

- Libros con más de 4 copias

```
Context: db:get("XML")
fichero*
1 for $l in /biblioteca/libros/libro
2 where $l/num_copias > 4
3 return $l/titulo
4
```

Resultado

2 Resultados, 50 b

```
<titulo>1984</titulo>
<titulo>Fundación</titulo>
```

Explicación

for \$l in /biblioteca/libros/libro

Recorre todos los libros.

where \$l/num_copias > 4

- Aplica un filtro: solo pasan los libros cuyo <num_copias> sea mayor que 4.
- En nuestro XML:
 - 1984 → 6
 - Fundación → 5

return \$l/titulo

- Devuelve el título de los libros que han pasado el filtro.
 - Resultado:
 - 1984
 - Fundación
-
- **Usuarios con penalización grave (id_penalizacion = 2)**

```
Context: db:get("XML")
fichero*
1 for $u in /biblioteca/usuarios/usuario
2 where $u/id_penalizacion = 2
3 return $u/nombre_user
4
```

Resultado

1 Resultado, 33 b

`<nombre_user>Markel</nombre_user>`

Explicación

for \$u in /biblioteca/usuarios/usuario

Recorre todos los <usuario>.

where \$u/id_penalizacion = 2

- Filtra solo los usuarios cuyo <id_penalizacion> sea 2.
- En nuestro XML:
 - Markel tiene id_penalizacion 2.

return \$u/nombre_user

- Devuelve el nombre del usuario filtrado.
- Resultado:
 - Markel

- **Libros y sus autores (JOIN)**



```
Context: db:get("XML")
fichero*
1 for $l in /biblioteca/libros/libro
2 let $autorID := /biblioteca/libros_autores/relacion[@id_libro = $l/@id_libro]/@id_autor
3 let $autor := /biblioteca/autores/autor[@id_autor = $autorID]
4 return <libro>
5   <titulo>{ $l/titulo/text() }</titulo>
6   <autor>{ $autor/nombre/text() } { $autor/apellidos/text() }</autor>
7 </libro>
```

Resultado

3 Resultados, 210 b

```
<libro><titulo>El Quijote</titulo><autor>Miguelde Cervantes</autor></libro>
<libro><titulo>1984</titulo><autor>GeorgeOrwell</autor></libro>
<libro><titulo>Fundación</titulo><autor>IsaacAsimov</autor></libro>
```

Explicación

for \$l in /biblioteca/libros/libro

- Recorre cada libro y lo guarda en \$l.

let \$autorID := /biblioteca/libros_autores/relacion[@id_libro = \$l/@id_libro]/@id_autor

- Busca en <libros_autores> la <relacion> cuyo @id_libro coincida con el @id_libro del libro actual \$l.
- De esa relación, coge el atributo @id_autor.
- Guarda ese valor en la variable \$autorID.
- Ejemplo:
 - Para “1984”, \$autorID será 2.

let \$autor := /biblioteca/autores/autor[@id_autor = \$autorID]

- Busca en <autores> el <autor> cuyo @id_autor coincida con \$autorID.
- Guarda ese nodo <autor> en \$autor.

return <libro> ... </libro>



- Construye un nuevo XML para cada libro:
 - <titulo> → texto del título del libro.
 - <autor> → nombre + apellidos del autor.

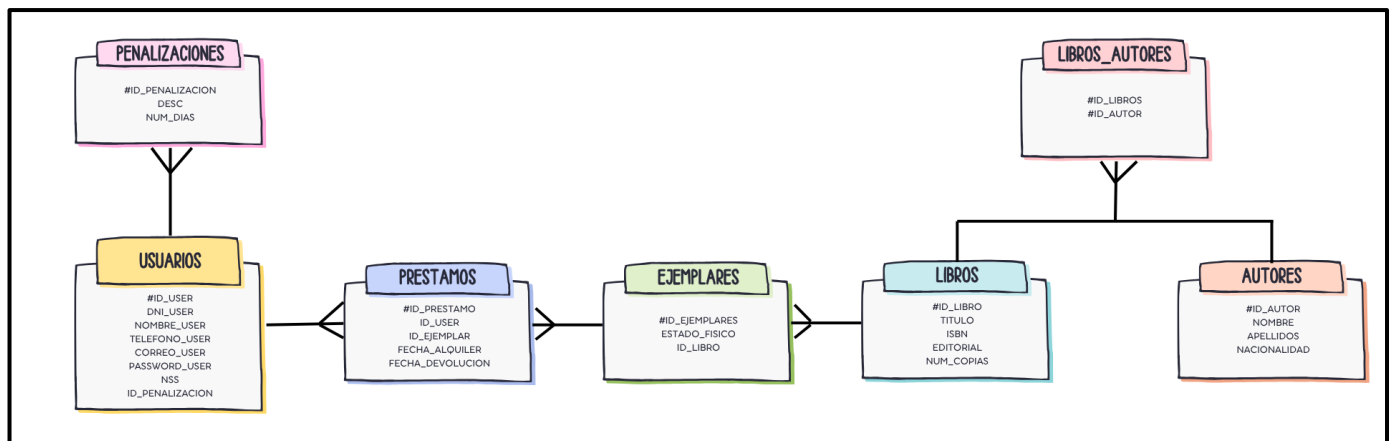
Bases de datos

La biblioteca de Muskiz está desarrollando un nuevo proyecto en el que nos han pedido que, dentro de la aplicación que estamos creando en Java, realicemos una conexión a una base de datos para poder gestionar y almacenar la información de forma más eficiente. Gracias a este proyecto, estamos aprendiendo cómo una aplicación puede interactuar con una base de datos mediante diferentes consultas y operaciones, como insertar, modificar, eliminar y consultar información desde el propio programa.

Además, en la asignatura de Lenguaje de Marcas estamos trabajando las similitudes entre XQuery y SQL. Hemos comprobado que ambos lenguajes se utilizan para realizar consultas sobre datos, aunque SQL está orientado principalmente a bases de datos relacionales y XQuery a documentos XML. También comparten características similares, como la selección de información, el filtrado de resultados y la organización de los datos obtenidos mediante consultas.



Diagrama E/R de nuestra biblioteca



Desde el inicio del proyecto hemos trabajado con las **mismas tablas base**, manteniendo una estructura coherente en todo el desarrollo. Para representar sus relaciones, hemos incorporado el **diagrama Entidad–Relación**, que muestra de forma clara cómo se conectan las tablas y facilita la comprensión del modelo de datos utilizado.

3 – OPERACIONES: CONSULTAS, MODIFICACIONES, BORRADO E INSERCCIONES

En este apartado, una vez realizado el código de la conexión a la base de datos, se realizan las operaciones de consultas, modificaciones, inserciones, y borrados DE TODAS LAS TABLAS (LIBROS, AUTORES, USUARIOS, PRESTAMOS, EJEMPLARES, PENALIZACIONES).

Todas ellas, son operaciones fundamentales que realiza una aplicación escrita en el lenguaje Java para realizar peticiones con una base de datos utilizando el lenguaje SQL.

En el caso de la gestión de nuestra biblioteca, estas operaciones por ejemplo se pueden utilizar para:

SELECT: mostrar información al usuario.

INSERT: guardar nueva información por primera vez.

UPDATE: cuando la información cambia y se necesita actualizarla sin borrar todo el registro.

DELETE: Se utiliza para borrar datos que ya no son útiles, necesarios o válidos.

TABLA LIBROS

SELECT:

```
1 String sql = "SELECT id_libro, titulo, isbn, editorial, genero, num_copias FROM libros";
2
```

INSERT:

```
1 String sql = "INSERT INTO libros (id_libro, titulo, isbn, editorial, genero, num_copias) VALUES (?, ?, ?, ?, ?, ?)";
2
```




UPDATE:

```
1 String sql = "UPDATE libros SET editorial = ? WHERE id_libro = ?";  
2
```

DELETE:

```
1 String sql = "DELETE FROM libros WHERE id_libro = ?";  
2
```

TABLA AUTOR

SELECT:

```
1 String sql = "SELECT id_autor, nombre, apellidos, nacionalidad FROM autores";  
2
```

INSERT:

```
1 String sql = "INSERT INTO autores (id_autor, nombre, apellidos, nacionalidad) VALUES (?, ?, ?, ?)";  
2
```



UPDATE:

```
1 String sql = "UPDATE autores SET nacionalidad = ? WHERE id_autor = ?";
```

DELETE:

```
1 String sql = "DELETE FROM autores WHERE id_autor = ?";
```

TABLA USUARIOS

SELECT:

```
1 String sql = "SELECT * FROM usuarios";
```

INSERT:

```
1 String sql = "INSERT INTO usuarios (id_user, dni_user, nombre_user, telefono_user, password_user, nss, id_penalizacion) VALUES (?, ?, ?, ?, ?, ?, ?)";
```



UPDATE:

```
1 String sql = "UPDATE usuarios SET telefono_user = ? WHERE id_user = ?";
```

DELETE:

```
1 String sql = "DELETE FROM usuarios WHERE id_user = ?";
```

TABLA EJEMPLAR

SELECT:

```
1 String sql = "SELECT id_ejemplar, estado_fisico, id_libro FROM ejemplares";
```

INSERT:

```
1 String sql = "INSERT INTO ejemplares (id_ejemplar, estado_fisico, id_libro) VALUES (?, ?, ?)";
```



UPDATE:

```
1 String sql = "UPDATE ejemplares SET estado_fisico = ? WHERE id_ejemplar = ?";
```

DELETE:

```
1 String sql = "DELETE FROM ejemplares WHERE id_ejemplar = ?";
```

TABLA PRESTAMOS

SELECT:

```
1 String sql = "SELECT * FROM prestamos";
```

INSERT

```
1 String sql = "INSERT INTO prestamos (id_prestamo, fecha_alquiler, fecha_devolucion, id_ejemplar, id_user) VALUES (?, ?, ?, ?, ?)";
```



UPDATE:

```
1 String sql = "UPDATE prestamos SET fecha_devolucion = ? WHERE id_prestamo = ?";
```

DELETE:

```
1 String sql = "DELETE FROM prestamos WHERE id_prestamo = ?";
```

Apartado de Lenguaje de Marcas

- **Similitudes entre XQuery y SQL**

Aunque XQuery y SQL pertenecen a entornos tecnológicos diferentes, ambos lenguajes comparten una base conceptual sólida orientada a la **consulta, filtrado y manipulación de datos estructurados**. SQL se utiliza tradicionalmente en sistemas de bases de datos relacionales, donde la información se organiza en tablas, filas y columnas. Por su parte, XQuery está diseñado para trabajar con documentos XML, cuya estructura es jerárquica y basada en nodos.

1. Ambos son lenguajes de consulta

- SQL consulta tablas y filas.
- XQuery consulta nodos XML.

En ambos casos, el objetivo es **extraer información específica** de un conjunto de datos estructurados.

2. Ambos usan filtros para seleccionar datos

- SQL utiliza WHERE.
- XQuery también utiliza where.

3. Ambos permiten unir datos de distintas “tablas”

- SQL usa JOIN.
- XQuery usa combinaciones de for, let y rutas XML para unir nodos.

Ambos resuelven relaciones **1:1**, **1:N** y **N:M**.



4. Ambos permiten ordenar, agrupar y transformar datos

SQL:

ORDER BY, GROUP BY

XQuery:

order by, group by

Además, XQuery puede **generar XML o HTML**, algo que SQL no puede hacer.

5. Ambos usan variables

- SQL usa alias (AS, “ ”).
- XQuery usa variables (let \$x := ...).

6. Ambos permiten funciones

SQL tiene funciones como:

- MAX(), COUNT(), AVG()

XQuery también:

- max(), count(), avg(), distinct-values(), string-length()

- **¿XQuery se parece más a SQL o a PL/SQL?**

XQuery se parece más a SQL en la lógica de consulta

Porque ambos:

- Seleccionan datos
- Filtran (where)
- Ordenan (order by)
- Agrupan
- Hacen “joins”



- Usan funciones (count(), avg(), etc.)

Pero XQuery se parece más a PL/SQL en la forma de escribirlo

Porque:

- Usa **variables** (let \$x := ...)
- Tiene **estructuras más largas**
- Permite **construir resultados complejos**
- No es solo una consulta: puede **generar XML**, igual que PL/SQL puede generar texto o estructuras

Ejercicios de ejemplos

(La base de datos en XML tiene menos información que la de SQL, así que es normal que algunas consultas devuelvan menos resultados o no coincidan exactamente con los resultados que den las consultas de SQL.)

- **Ejercicio 1 — Seleccionar todos los títulos**

XQUERY

```
Context: db:get("XML")
1 for $l in /biblioteca/libros/libro
2 return $l/titulo
```

Resultado

3 Resultados, 79 b

```
<titulo>El Quijote</titulo>
<titulo>1984</titulo>
<titulo>Fundación</titulo>
```

Explicación

```
for $l in /biblioteca/libros/libro
```

- Recorre todos los nodos <libro> dentro de /biblioteca/libros/.
- Cada libro se guarda temporalmente en la variable \$l.

```
return $l/titulo
```

- Devuelve el contenido del nodo <titulo> de cada libro.

SQL

```
SELECT titulo  
FROM libros;
```

Resultado

```
TITULO  
-----  
Cien años de soledad  
Don Quijote  
1984  
El hobbit  
Orgullo y prejuicio  
Crimen y castigo  
Harry Potter y la piedra  
El código Da Vinci  
Los juegos del hambre  
It  
  
10 filas seleccionadas.
```

Ambos recorren una colección de datos y devuelven una columna/campo.

- SQL recorre filas de una tabla.
- XQuery recorre nodos <libro>.

• Ejercicio 2 — Filtrar por condición (WHERE)

XQUERY

```
Context: db:get("XML")  
1 for $l in /biblioteca/libros/libro  
2 where $l/num_copias > 4  
3 return $l/titulo
```



Resultado

📄 🏠 🔍 2 Resultados, 50 b

```
<titulo>1984</titulo>
<titulo>Fundación</titulo>
```

Explicación

```
for $l in /biblioteca/libros/libro
```

Recorre todos los nodos <libro> dentro del XML. Cada libro se guarda en la variable \$l.

```
where $l/num_copias > 4
```

Filtra los libros y se queda solo con aquellos cuyo <num_copias> es mayor que 4.

```
return $l/titulo
```

Devuelve el título de cada libro que cumple la condición.

SQL

```
SELECT titulo
FROM libros
WHERE num_copias > 4;
```

Resultado

```
TITULO
-----
Cien años de soledad
Don Quijote
1984
El hobbit
Orgullo y prejuicio
Crimen y castigo
Harry Potter y la piedra
El código Da Vinci
Los juegos del hambre
It
10 filas seleccionadas.
```

El uso del filtro es exactamente el mismo:

- where en XQuery funciona igual que WHERE en SQL. Ambos devuelven solo los elementos que cumplen la condición.

• Ejercicio 3 — Seleccionar usuarios con penalización

XQUERY

```
Context: db:get("XML")
1 for $u in /biblioteca/usuarios/usuario
2 where $u/id_penalizacion = 2
3 return $u/nombre_user
```

Resultado

1 Resultado, 33 b
<nombre_user>Markel</nombre_user>

Explicación

```
for $u in /biblioteca/usuarios/usuario
```

Recorre todos los nodos <usuario> dentro del XML. Cada usuario se guarda en la variable \$u.

```
where $u/id_penalizacion = 2
```

Filtra los usuarios y se queda solo con aquellos cuyo <id_penalizacion> es igual a 2.

```
return $u/nombre_user
```

Devuelve el nombre del usuario que cumple la condición.



SQL

```
SELECT nombre_user  
FROM usuarios  
WHERE id_penalizacion = 2;
```

Resultado

```
NOMBRE_USER  
-----  
Javier Ruiz  
  
1 fila seleccionada.
```

Recorrer datos → Filtrar → Devolver un valor

XQuery lo hace con for ... where ... return

SQL lo hace con SELECT ... FROM ... WHERE



- Ejercicio 4 — JOIN entre tablas (Libros + Autores)

XQUERY

```
Context: db:get("XML")
fichero*
1 for $l in /biblioteca/libros/libro
2 let $autorID := /biblioteca/libros_autores/relacion[@id_libro = $l/@id_libro]/@id_autor
3 let $autor := /biblioteca/autores/autor[@id_autor = $autorID]
4 return <resultado>
5   <titulo>{ $l/titulo/text() }</titulo>
6   <autor>{ $autor/nombre/text() }</autor>
7 </resultado>
```

Resultado

3 Resultados, 210 b

```
<resultado><titulo>El Quijote</titulo><autor>Miguel</autor></resultado>
<resultado><titulo>1984</titulo><autor>George</autor></resultado>
<resultado><titulo>Fundación</titulo><autor>Isaac</autor></resultado>
```

Explicación

```
for $l in /biblioteca/libros/libro
```

Recorre todos los libros. Cada <libro> se guarda en la variable \$l.

```
let $autorID := /biblioteca/libros_autores/relacion[@id_libro =
$l/@id_libro]/@id_autor
```

Aquí ocurre el equivalente al **JOIN intermedio**:

- Busca en <relacion> la entrada donde id_libro coincide con el libro actual.
- Extrae el atributo id_autor.

```
let $autor := /biblioteca/autores/autor[@id_autor = $autorID]
```



Ahora hace el segundo JOIN:

- Busca el <autor> cuyo id_autor coincide con el obtenido antes.

```
<resultado>
<titulo>...</titulo>
<autor>...</autor>
</resultado>
```

XQuery permite **crear estructuras XML nuevas** como salida.

SQL

```
SELECT l.titulo, a.nombre
FROM libros l
JOIN libros_autores la ON la.id_libro = l.id_libro
JOIN autores a ON a.id_autor = la.id_autor;
```

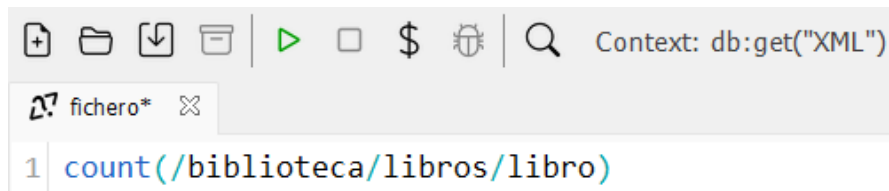
Resultado

TITULO	NOMBRE
Cien años de soledad	Gabriel
Don Quijote	Miguel
1984	George
El hobbit	J.R.R.
Orgullo y prejuicio	Jane
Crimen y castigo	Fyodor
Harry Potter y la piedra	J.K.
El código Da Vinci	Dan
Los juegos del hambre	Suzanne
It	Stephen
10 filas seleccionadas.	

Concepto	XQUERY	SQL
Tipo de datos	XML	Tablas
JOIN	Se hace con let + filtros	Se hace con JOIN
Relación	Comparando atributos	Comparando columnas
Salida	XML personalizado	Tabla

- **Ejercicio 5 — Contar elementos**

XQUERY



Resultado

1 Resultado, 1 b

3

Explicación

/biblioteca/libros/libro

Selecciona todos los nodos <libro> dentro del XML.

count(...)

Cuenta cuántos nodos <libro> existen.

SQL

SELECT COUNT(*)

FROM libros;

Las funciones agregadas existen en ambos lenguajes.

- count() funciona igual en SQL y XQuery. Ambos devuelven el número total de elementos.

Concepto	XQUERY	SQL
Qué cuenta	Nodos <libro>	Filas de la tabla libros
Función	Count()	COUNT()
Resultado	Un número	Un número
Estructura	XML	Tabla

- Ejercicio 6 — Ordenar resultados

XQUERY

```
Context: db:get("XML")
1 for $l in /biblioteca/libros/libro
2 order by $l/titulo
3 return $l/titulo
```

Resultado

3 Resultados, 79 b

```
<titulo>1984</titulo>
<titulo>El Quijote</titulo>
<titulo>Fundación</titulo>
```

Explicación

```
for $l in /biblioteca/libros/libro
```

Recorre todos los nodos <libro> del XML. Cada libro se guarda en la variable \$l.



order by \$l/titulo

Ordena los libros según el contenido del nodo <titulo>. Por defecto, el orden es ascendente (A-Z).

return \$l/titulo

Devuelve el título de cada libro ya ordenado.

SQL

```
SELECT TITULO FROM LIBROS ORDER BY TITULO
```

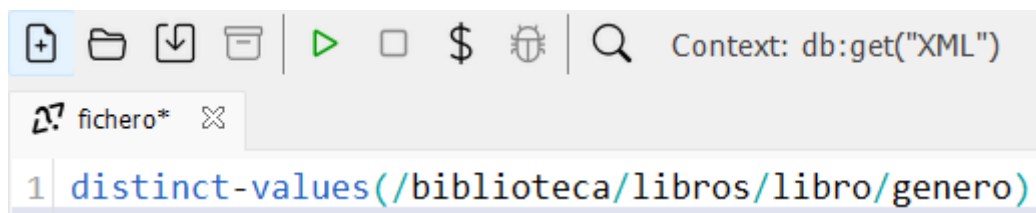
Resultado

```
TITULO
-----
Cien años de soledad
Crimen y castigo
Don Quijote
El código Da Vinci
El hobbit
Harry Potter y la piedra
It
Los juegos del hambre
Orgullo y prejuicio
1984
10 filas seleccionadas.
```

Concepto	XQUERY	SQL
Qué recorre	Nodos <libro>	Filas de la tabla libros
Como ordena	order by \$l/titulo	ORDER BY titulo
Orden por defecto	Ascendente	Ascendente
Qué devuelve	Nodo <titulo>	Columna titulo

- Ejercicio 7 — Agrupar o eliminar duplicados

XQUERY



Resultado

3 Resultados, 37 b

Clásico
Distopía
Ciencia Ficción

Explicación

/biblioteca/libros/libro/genero



Selecciona todos los nodos <genero> de todos los libros.

```
distinct-values(...)
```

Elimina los valores repetidos y devuelve solo los géneros únicos.

SQL

```
SELECT DISTINCT GENERO FROM LIBROS;
```

Resultado

```
GENERO
-----
Ciencia ficción
Clásico
Distopía
Drama
Fantasía
Misterio
Realismo mágico
Romántica
Terror

9 filas seleccionadas.
```

Concepto	XQUERY	SQL
Qué recorre	Nodos <genero>	Filas de la tabla libros
Cómo elimina duplicados	distinct-values()	DISTINCT
Qué devuelve	Valores únicos XML	Valores únicos en tabla
Estructura	XML	Tabla relacional

- **Ejercicio 8 — Subconsulta: libros con más copias que la media**

XQUERY

```
Context: db:get("XML")

1 let $media := avg(/biblioteca/libros/libro/num_copias)
2 for $l in /biblioteca/libros/libro
3 where $l/num_copias > $media
4 return $l/titulo
```

Resultado

1 Resultado, 21 b

<titulo>1984</titulo>

Explicación

```
let $media := avg(/biblioteca/libros/libro/num_copias)
```

- Selecciona todos los <num_copias> del XML.
- Calcula la media con avg().
- Guarda el resultado en la variable \$media.

```
for $l in /biblioteca/libros/libro
```

Cada libro se guarda en \$l.

```
where $l/num_copias > $media
```

Solo pasan los libros cuyo número de copias es mayor que la media calculada antes.

```
return $l/titulo
```

Devuelve solo los libros que cumplen la condición



SQL

```
SELECT TITULO FROM LIBROS WHERE NUM_COPIAS >
(SELECT AVG (NUM_COPIAS) FROM LIBROS);|
```

Resultado

```
TITULO
-----
El hobbit
Harry Potter y la piedra
El código Da Vinci
Los juegos del hambre

4 filas seleccionadas.
```

Concepto	XQUERY	SQL
Cálculo de la media	avg()	AVG()
Subconsulta	let \$media := ...	Subconsulta dentro de WHERE
Filtro	where \$l/num_copias > \$media	WHERE num_copias > (...)
Qué recorre	Nodos XML	Filas de tabla

- Ejercicio 9 — Agrupación avanzada: contar libros por género

XQUERY

```
Context: db:get("XML")

1 for $g in distinct-values(/biblioteca/libros/libro/genero)
2 let $cantidad := count(/biblioteca/libros/libro[genero = $g])
3 return <genero nombre="{ $g}" total="{ $cantidad}"/>
```

Resultado

3 Resultados, 124 b

```
<genero nombre="Clásico" total="1"/>
<genero nombre="Distopía" total="1"/>
<genero nombre="Ciencia Ficción" total="1"/>
```

Explicación

`distinct-values(/biblioteca/libros/libro/genero)`

Esto es equivalente a `SELECT DISTINCT genero` en SQL. Devuelve una lista de géneros únicos.

`for $g in distinct-values(...)`

Cada género se guarda en la variable `$g`.

`let $cantidad := count(/biblioteca/libros/libro[genero = $g])`

- Busca todos los `<libro>` cuyo `<genero>` coincide con `$g`.
- Los cuenta con `count()`.

`<genero nombre="{ $g}" total="{ $cantidad}"/>`

Construye un XML con el resultado



SQL

```
SELECT genero, COUNT(*)
```

```
FROM libros
```

```
GROUP BY genero;
```

Resultado

Concepto	XQUERY	SQL
Obtener valores únicos	distinct-values()	DISTINCT
Agrupar	No existe GROUP BY → se simula con for + count()	GROUP BY
Contar elementos	count()	COUNT()
Salida	XML personalizado	Tabla

- Ejercicio 10 — JOIN múltiple con transformación de salida

XQUERY

```
Context: db:get("XML")

1 for $p in /biblioteca/prestamos/prestamo
2 let $user := /biblioteca/usuarios/usuario[@id_user = $p/id_user]
3 let $ej := /biblioteca/ejemplares/ejemplar[@id_ejemplar = $p/id_ejemplar]
4 let $libro := /biblioteca/libros/libro[@id_libro = $ej/id_libro]
5 return <prestamo>
6     <usuario>{ $user/nombre_user/text() }</usuario>
7     <titulo>{ $libro/titulo/text() }</titulo>
8     <fecha>{ $p/fecha_prestamo/text() }</fecha>
9 </prestamo>
```

Resultado

3 Resultados, 238 b

```
<prestamo><usuario>Ainur</usuario><titulo>El Quijote</titulo><fecha/></prestamo>
<prestamo><usuario>Markel</usuario><titulo>1984</titulo><fecha/></prestamo>
<prestamo><usuario>Enol</usuario><titulo>Fundación</titulo><fecha/></prestamo>
```

Explicación

for \$p in /biblioteca/prestamos/prestamo

Cada <prestamo> se guarda en \$p.

let \$user := /biblioteca/usuarios/usuario[@id_user = \$p/id_user]

Busca el <usuario> cuyo id_user coincide con el del préstamo.

let \$ej := /biblioteca/ejemplares/ejemplar[@id_ejemplar = \$p/id_ejemplar]

Busca el <ejemplar> prestado.

let \$libro := /biblioteca/libros/libro[@id_libro = \$ej/id_libro]

Busca el <libro> asociado al ejemplar.



SQL

```
SELECT u.nombre_user, l.titulo, p.fecha_alquiler
FROM prestamos p
JOIN usuarios u ON u.id_user = p.id_user
JOIN ejemplares e ON e.id_ejemplar = p.id_ejemplar
JOIN libros l ON l.id_libro = e.id_libro;
```

Resultado

NOMBRE_USER	TITULO	FECHA_AL
Carlos Pérez	Cien años de soledad	01/11/25
Javier Ruiz	Cien años de soledad	03/11/25
Andrés Martín	Don Quijote	05/11/25
David Romero	Don Quijote	06/11/25
Elena Navarro	1984	07/11/25
Carlos Pérez	1984	02/11/25
María López	El hobbit	04/11/25
Lucía Fernández	El hobbit	05/11/25
Sara Torres	Orgullo y prejuicio	06/11/25
Laura Gómez	Orgullo y prejuicio	07/11/25

10 filas seleccionadas.

Concepto	XQUERY	SQL
Tipo de datos	XML	Tablas
JOIN	let + filtros	JOIN ... ON
Relación	Comparando atributos	Comparando columnas
Salida	XML personalizado	Tabla

- **Ejercicio 11 — Obtener el libro más prestado**

XQUERY

```
Context: db:get("XML")

1 let $libros := /biblioteca/libros/libro
2 let $prestamos := /biblioteca/prestamos/prestamo
3 for $l in $libros
4 let $total := count($prestamos[id_libro = $l/@id_libro])
5 order by $total descending
6 return <masPrestado titulo="{ $l/titulo }" total="{ $total }"/>
```

Resultado

3 Resultados, 130 b

```
<masPrestado titulo="El Quijote" total="0"/>
<masPrestado titulo="1984" total="0"/>
<masPrestado titulo="Fundación" total="0"/>
```

Explicación

```
let $libros := /biblioteca/libros/libro
```

- Selecciona todos los nodos <libro>.
- Los guarda en la variable \$libros.

```
let $prestamos := /biblioteca/prestamos/préstamo
```

- Selecciona todos los nodos <prestamo>.
- Los guarda en \$prestamos.
- Es la lista completa de préstamos realizados.

```
for $l in $libros
```

- Recorre cada libro uno por uno.
- \$l representa el libro actual del bucle.



```
let $total := count($prestamos[id_libro = $l/@id_libro])
```

- Busca dentro de \$prestamos todos los préstamos cuyo id_libro coincida con el del libro actual.
- count() cuenta cuántos hay.
- El resultado se guarda en \$total.

Esto es equivalente a un COUNT(*) con GROUP BY en SQL.

```
order by $total descending
```

- Ordena los libros según \$total.
- descending significa de mayor a menor.
- El libro más prestado quedará primero.

```
return <masPrestado titulo="{ $l/titulo}" total="{ $total}"/>
```

Crea un nodo <masPrestado> por cada libro.

Incluye:

- El título del libro
- El total de préstamos



SQL

```
SELECT l.titulo,  
       COUNT(p.id_libro) AS total  
FROM libros l  
LEFT JOIN prestamos p  
       ON p.id_libro = l.id_libro  
GROUP BY l.titulo  
ORDER BY total DESC;
```

Concepto	XQUERY	SQL
Tipo de datos	XML	Tablas
Cómo recorre los elementos	for \$l in \$libros	FROM libros
Cómo relaciona datos (JOIN)	let + filtros sobre atributos	JOIN ... ON
Cómo cuenta préstamos	count(\$prestamos[id_libro = \$l/@id_libro])	COUNT(p.id_libro)
Cómo agrupa	No existe GROUP BY → se simula con for + count()	GROUP BY titulo
Cómo ordena	order by \$total descending	ORDER BY total DESC



Conclusión

A lo largo de este proyecto hemos comprobado que, aunque existen distintos lenguajes de consulta, todos comparten una base conceptual común. XQuery, por ejemplo, está diseñado para trabajar con documentos XML, pero su forma de consultar, filtrar, ordenar y relacionar datos guarda un parecido muy fuerte con SQL y PL/SQL.

XQuery se parece a SQL porque:

- Ambos recorren colecciones de datos (SQL → filas; XQuery → nodos XML).
- Ambos filtran con `where` / `WHERE`.
- Ambos ordenan con `order by`.
- Ambos cuentan elementos con `count()` / `COUNT()`.
- Ambos permiten obtener valores únicos o agrupados.
- Ambos pueden relacionar datos (SQL con `JOIN`, XQuery con `let` + filtros).

XQuery se parece a PL/SQL porque:

- Usa **variables** (`let`) igual que PL/SQL.
- Permite **bloques lógicos** y operaciones más complejas que una simple consulta.
- Puede **construir resultados personalizados**, igual que PL/SQL genera estructuras o mensajes.
- Permite combinar datos de varias “tablas” XML con lógica adicional.

Gracias a esta combinación, los empleados pueden:

- Consultar préstamos, usuarios y ejemplares de forma rápida.
- Generar informes personalizados.
- Detectar libros más prestados, géneros más populares o usuarios con penalizaciones.
- Mantener la información sincronizada entre la base de datos y los documentos XML.

Entornos de desarrollo

En este proyecto, como equipo de desarrollo de la biblioteca, se nos ha solicitado mejorar y ampliar el trabajo previamente realizado sobre el sistema de gestión de préstamos. Para ello, se ha actualizado el **diagrama de clases** con nuevas relaciones y estructuras que reflejan con mayor precisión el funcionamiento real del sistema. Además, se han creado los **diagramas de objetos** correspondientes, mostrando ejemplos concretos de las instancias y vínculos que intervienen en el proceso de préstamo.

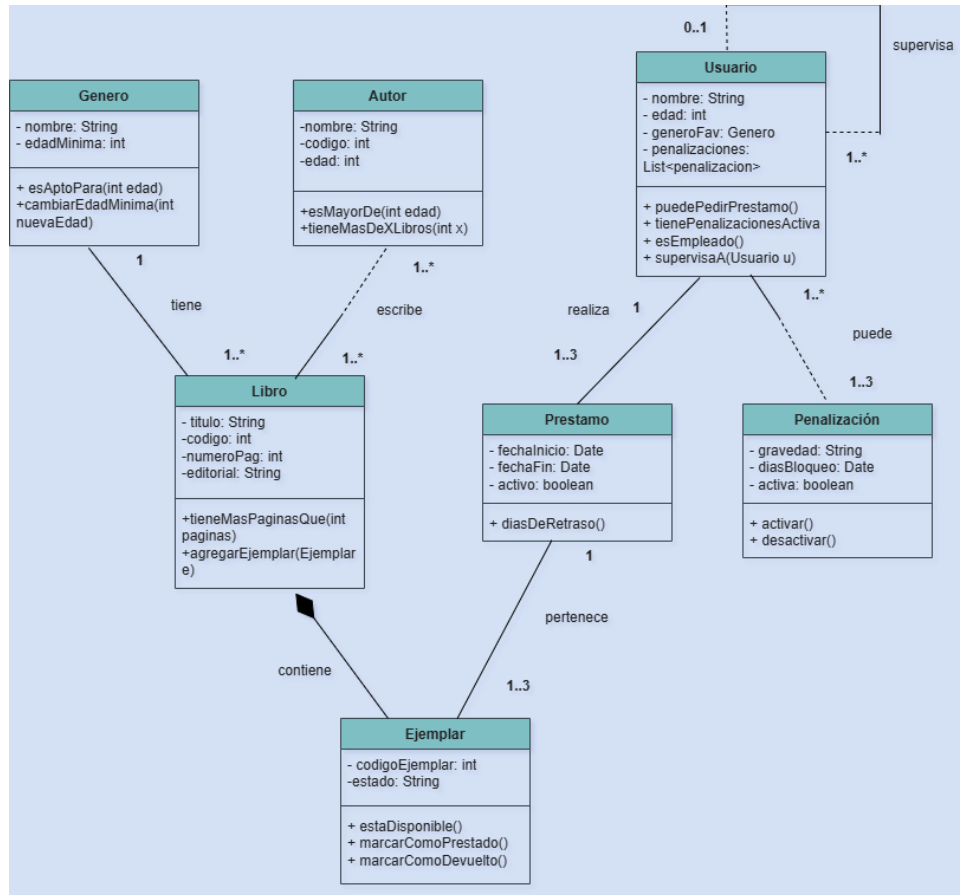
Con el objetivo de facilitar la comprensión del funcionamiento de la aplicación a los usuarios y al personal de la biblioteca, se ha elaborado también un **diagrama de casos de uso**, junto con los **diagramas de secuencia** que describen el flujo de interacción entre actores y componentes del sistema. Todo el trabajo se ha desarrollado utilizando la herramienta DRAW.IO, que incorpora herramientas adecuadas para la creación de diagramas UML.

Finalmente, se incluye un **enlace al repositorio de GitHub**, donde pueden consultarse las clases Java implementadas y verificarse el uso correcto de un **sistema de control de versiones (SCV)** durante el desarrollo.



Diagrama de clases

Hemos actualizado el diagrama de clases que hicimos anteriormente para que encaje mejor con cómo está realmente programada la aplicación. Básicamente, el diagrama ahora refleja tal cual la estructura del código.



Explicación

En este diagrama existen 7 clases con sus respectivos métodos.

- 1º Clase Género:** Un género lo tiene uno o muchos libros y un libro tiene un solo género.
- 2º Clase Autor:** Un libro puede ser escrito por uno o mas autores, y un autor puede escribir uno o muchos libros.
- 3º Clase Libro:** En esta clase hemos agregado una composición fuerte con la clase ejemplar, ya que, si un libro deja de existir, automáticamente los ejemplares desaparecen.
- 4º Clase Ejemplar:** Un ejemplar pertenece a un solo préstamo ya que un ejemplar es único, no hay más de uno con un mismo código. Y un préstamo puede tener de uno hasta 3 ejemplares, que es lo máximo que nos permite la biblioteca.



5º Clase Préstamo: Un préstamo lo realiza un solo usuario, y un usuario puede tener de uno hasta 3 préstamos, ya que es lo que nos permite nuestra biblioteca.

6º Clase Penalización: Una penalización la puede tener uno o muchos usuarios y como máximo solo puede tener 3 penalizaciones.

7º Clase Usuario: En la clase usuario existe una relación reflexiva, ya que un usuario puede ser un empleado, es decir, que puede supervisar a uno o varios usuarios.

METODOS

Hemos definido los métodos de cada clase basándonos en la función que cumplen dentro de la aplicación y en la lógica necesaria para que el sistema de biblioteca funcione correctamente. A continuación, se muestra el **código Java correspondiente al diagrama de clases**, incluyendo todas las clases y sus métodos implementados.

CLASE GÉNERO

```
public boolean esAptoPara(int edad) {  
    return edad >= edadMinima;  
}  
  
public void cambiarEdadMinima(int nuevaEdad) {  
    this.edadMinima = nuevaEdad;  
}
```

CLASE AUTOR

```
public boolean esMayorDe(int edad) {  
    return this.edad > edad;  
}  
  
public boolean tieneMasDeXLibros(int x) {  
    return libros.size() > x;  
}
```



CLASE LIBRO

```
public boolean tieneMasPaginasQue(int paginas) {  
    return numeroPaginas > paginas;  
}  
  
public void agregarEjemplar(Ejemplar e) {  
    ejemplares.add(e); }  
}
```

CLASE EJEMPLAR

```
public boolean estaDisponible() {  
    return !prestado;  
}  
  
public void marcarComoPrestado() {  
    prestado = true;  
}  
  
public void marcarComoDevuelto() {  
    prestado = false;  
}  
}
```

CLASE USUARIO

```
public boolean puedePedirPrestamo() {  
    return prestamosActivos.size() < 3 && !tienePenalizacionesActivas();  
}  
  
public boolean tienePenalizacionesActivas() {  
    for (Penalizacion p : penalizaciones) {  
        if (p.isActiva()) return true;  
    }  
    return false;  
}  
  
public boolean esEmpleado() {  
    return !supervisados.isEmpty();  
}  
}
```



```
public void supervisaA(Usuario u) {  
    supervisados.add(u);  
}  
}
```

CLASE PRÉSTAMO

```
public int diasDeRetraso() {  
    if (!estaAtrasado()) return 0;  
    return (int) java.time.temporal.ChronoUnit.DAYS.between(fechaFin, LocalDate.now());  
}
```

CLASE PENALIZACIÓN

```
public void activar() {  
    activa = true;  
}  
public void desactivar() {  
    activa = false;  
}
```

• INSTANCIAS DE LAS CLASES

```
import java.time.LocalDate;

public class Main {

    public static void main(String[] args) {

        // -----
        // 1. Crear géneros
        // -----
        Genero terror = new Genero("Terror", 16);
        Genero fantasia = new Genero("Fantasía", 0);

        // -----
        // 2. Crear autores
        // -----
        Autor king = new Autor("Stephen King", 1, 75);
        Autor lovecraft = new Autor("H.P. Lovecraft", 2, 46);

        // -----
        // 3. Crear libro con varios autores
        // -----
        Libro libroTerror = new Libro("Historias Oscuras", 101, 350, "DarkBooks", terror);
        libroTerror.agregarAutor(king);
        libroTerror.agregarAutor(lovecraft);

        // -----
        // 4. Crear ejemplares del libro
        // -----
        Ejemplar ej1 = new Ejemplar(1, "nuevo");
        Ejemplar ej2 = new Ejemplar(2, "usado");
```

```
Ejemplar ej3 = new Ejemplar(3, "nuevo");

libroTerror.agregarEjemplar(ej1);
libroTerror.agregarEjemplar(ej2);
libroTerror.agregarEjemplar(ej3);

// -----
// 5. Crear usuarios
// -----

Usuario usuario1 = new Usuario("Ainur", 19, fantasia);
Usuario empleado = new Usuario("Markel", 25, terror);

// Relación reflexiva: empleado supervisa a usuario1
empleado.supervisaA(usuario1);

// -----
// 6. Crear préstamos (máximo 3)
// -----

Prestamo p1 = new Prestamo(ej1, LocalDate.now(), LocalDate.now().plusDays(7));
Prestamo p2 = new Prestamo(ej2, LocalDate.now(), LocalDate.now().plusDays(7));
Prestamo p3 = new Prestamo(ej3, LocalDate.now(), LocalDate.now().plusDays(7));

usuario1.agregarPrestamo(p1);
usuario1.agregarPrestamo(p2);
usuario1.agregarPrestamo(p3);

// Intento de cuarto préstamo (NO debería permitirlo)
Ejemplar ej4 = new Ejemplar(4, "nuevo");
Prestamo p4 = new Prestamo(ej4, LocalDate.now(), LocalDate.now().plusDays(7));

if (usuario1.puedePedirPrestamo()) {
```

```
        usuario1.agregarPrestamo(p4);

        System.out.println("Préstamo 4 añadido (ERROR, no debería pasar)");
    } else {

        System.out.println("No se puede añadir el préstamo 4: límite de 3 alcanzado");
    }

    // -----
    // 7. Añadir penalización
    // -----

    Penalizacion pena = new Penalizacion("alta", 10);
    usuario1.agregarPenalizacion(pena);

    System.out.println("¿Usuario tiene penalizaciones activas? " +
usuario1.tienePenalizacionesActivas());

    // -----
    // 8. Cerrar un préstamo
    // -----

    p1.cerrarPrestamo();

    System.out.println("Préstamo 1 cerrado. ¿Ejemplar disponible? " + ej1.estaDisponible());

    // -----
    // 9. Comprobación de retraso
    // -----

    Prestamo prestamoAtrasado = new Prestamo(ej2, LocalDate.now().minusDays(10),
LocalDate.now().minusDays(5));

    usuario1.agregarPrestamo(prestamoAtrasado);

    System.out.println("¿Préstamo atrasado? " + prestamoAtrasado.estaAtrasado());

    System.out.println("Días de retraso: " + prestamoAtrasado.diasDeRetraso());

    // -----
```



```
// 10. Mostrar si el usuario es empleado

// -----

System.out.println("¿Markel es empleado? " + empleado.esEmpleado());
}
}
```

1. Crear géneros

```
Genero terror = new Genero("Terror", 16);
Genero fantasia = new Genero("Fantasía", 0);
```

Aquí simplemente creas **dos géneros literarios**:

- *Terror* → edad mínima 16
- *Fantasía* → apto para todas las edades

2. Crear autores

```
Autor king = new Autor("Stephen King", 1, 75);
Autor lovecraft = new Autor("H.P. Lovecraft", 2, 46);
```

Creas **dos autores reales**, cada uno con:

- nombre
- código
- edad

3. Crear libro con varios autores

```
Libro libroTerror = new Libro("Historias Oscuras", 101, 350, "DarkBooks", terror);
libroTerror.agregarAutor(king);
libroTerror.agregarAutor(lovecraft);
```



Aquí creas un **libro de terror** y le añades **dos autores**. Esto demuestra la relación **muchos a muchos** entre Libro ↔ Autor.

4. Crear ejemplares del libro

```
Ejemplar ej1 = new Ejemplar(1, "nuevo");  
Ejemplar ej2 = new Ejemplar(2, "usado");  
Ejemplar ej3 = new Ejemplar(3, "nuevo");
```

```
libroTerror.agregarEjemplar(ej1);  
libroTerror.agregarEjemplar(ej2);  
libroTerror.agregarEjemplar(ej3);
```

Un libro puede tener varias copias físicas. Aquí creas 3 ejemplares distintos del mismo libro.

5. Crear usuarios

```
Usuario usuario1 = new Usuario("Ainur", 19, fantasia);  
Usuario empleado = new Usuario("Markel", 25, terror);
```

Crear dos usuarios:

- **Ainur** → lector normal
- **Markel** → empleado (lo será cuando supervise a alguien)

Relación reflexiva (empleado-supervisa)

```
empleado.supervisaA(usuario1);
```


Esto activa la relación reflexiva:

- Un usuario puede supervisar a otros usuarios.
- Aquí Markel supervisa a Ainur → Markel pasa a ser empleado.

6. Crear préstamos (máximo 3)

```
Prestamo p1 = new Prestamo(ej1, LocalDate.now(), LocalDate.now().plusDays(7));  
Prestamo p2 = new Prestamo(ej2, LocalDate.now(), LocalDate.now().plusDays(7));  
Prestamo p3 = new Prestamo(ej3, LocalDate.now(), LocalDate.now().plusDays(7));
```

```
usuario1.agregarPrestamo(p1);  
usuario1.agregarPrestamo(p2);  
usuario1.agregarPrestamo(p3);
```

Creas **3 préstamos**, uno por cada ejemplar. El usuario puede tener **máximo 3 préstamos activos**, así que estos son válidos.

Intento de cuarto préstamo (NO permitido)

```
Ejemplar ej4 = new Ejemplar(4, "nuevo");  
Prestamo p4 = new Prestamo(ej4, LocalDate.now(), LocalDate.now().plusDays(7));
```

```
if (usuario1.puedePedirPrestamo()) {  
    usuario1.agregarPrestamo(p4);  
    System.out.println("Préstamo 4 añadido (ERROR, no debería pasar)");  
} else {  
    System.out.println("No se puede añadir el préstamo 4: límite de 3 alcanzado");  
}
```



Aquí demuestras la regla del sistema:

- Si ya tiene 3 préstamos → no puede pedir más.

El programa imprime:

No se puede añadir el préstamo 4: límite de 3 alcanzado

7. Añadir penalización

```
Penalizacion pena = new Penalizacion("alta", 10);  
usuario1.agregarPenalizacion(pena);
```

Creas una penalización grave y se la asignas al usuario. Esto afecta a su capacidad de pedir préstamos.

8. Cerrar un préstamo

```
p1.cerrarPrestamo();  
System.out.println("Préstamo 1 cerrado. ¿Ejemplar disponible? " + ej1.estaDisponible());
```

Cierras el préstamo → el ejemplar vuelve a estar disponible.

9. Comprobación de retraso

```
Prestamo prestamoAtrasado = new Prestamo(ej2, LocalDate.now().minusDays(10),  
LocalDate.now().minusDays(5));  
usuario1.agregarPrestamo(prestamoAtrasado);
```

Creas un préstamo que ya está vencido:

- Fecha inicio: hace 10 días
- Fecha fin: hace 5 días

El sistema detecta que está atrasado.



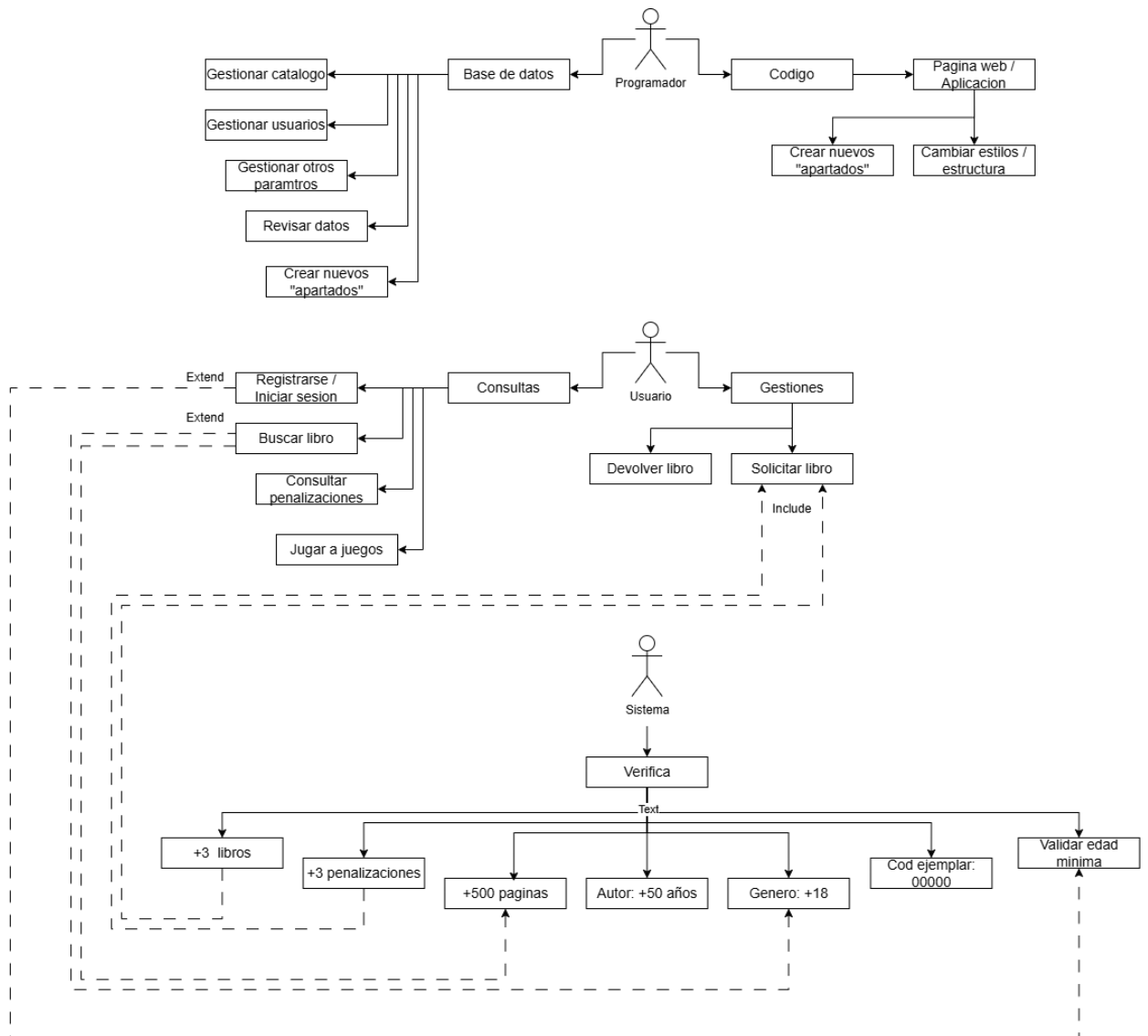
10. Comprobar si un usuario es empleado

```
System.out.println("¿Markel es empleado? " + empleado.esEmpleado());
```

Cómo supervisa a otro usuario → sí es empleado.

Diagrama de casos de uso

También hemos creado el diagrama de casos de uso para que cualquier persona que utilice la aplicación pueda entender de forma rápida qué acciones permite el sistema y cómo interactúan los usuarios con él. La idea es mostrar de manera sencilla las funciones principales y dejar claro cómo se utiliza la aplicación en el día a día.



Explicación

Representa a los actores que interactúan con el sistema y como sus funciones se relacionan entre sí.

En nuestra biblioteca, el programador (el que gestiona internamente la biblioteca) tiene acceso al sistema completo, que hemos dividido en 2 partes para verlo más claro:

- Base de datos: le permite gestionar datos ya almacenados, crear nuevas categorías de datos o simplemente verificarlos para buscar fallos, etc.
- Código: tanto la página web como la aplicación, tiene acceso completo al código de ambas y lo puede alterar de forma libre, ya sea para cambiar el estilo visual, la estructura, distribución de elementos, crear nuevas cosas, etc.

Por otro lado, el usuario tiene un acceso muy restringido a comparación, ya que no puede acceder al código o registros internos / ocultos (backend), solo a los elementos y estilos (frontend). Puesto de forma sencilla, solo puede hacer 2 distintas acciones:

- Consultas: no modifica el sistema ni sus datos, como registrarse / iniciar sesión, buscar información o jugar a los juegos disponibles.
- Gestiones: modifican parte del sistema, es este caso al pedir / devolver libro, se está creando / borrando un registro en la sección de préstamos y ejemplares.

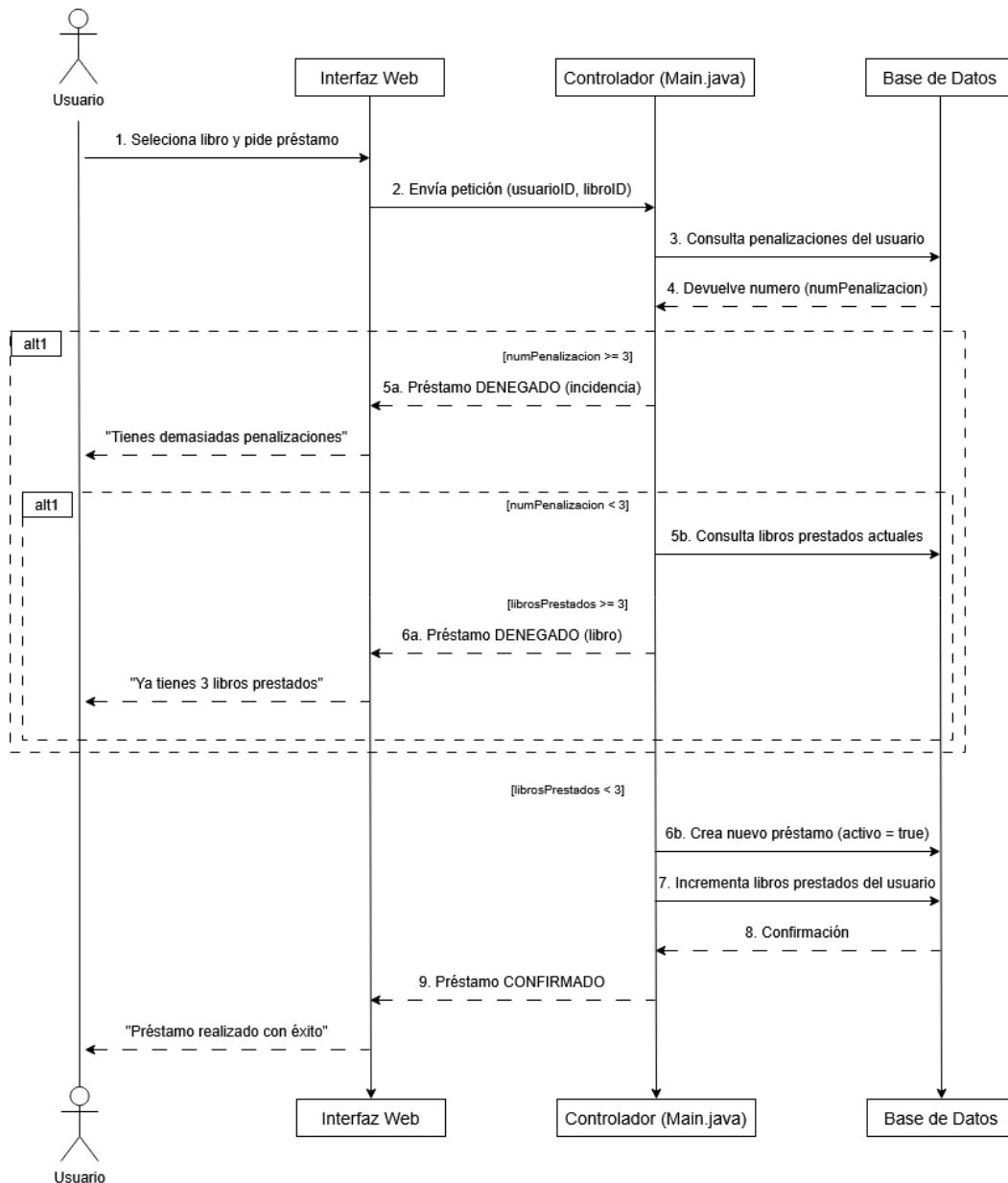
Como entidad no humana / automática, está el sistema, que puede verificar datos cuando se cumplen ciertos parámetros / acciones:

- con Include: cuando un usuario va a solicitar / devolver un libro, SIEMPRE (por eso es un include) verifica la cantidad de libros prestados y penalizaciones que tiene el usuario.

con Extend: al registrarse / iniciar sesión o al buscar un libro, se PUEDE (como es una POSIBILIDAD es un extend) verificar si tiene más de 500 páginas, género para mayores de edad o si cumple la edad mínima.

Diagrama de secuencia

También hemos creado los diagramas de secuencia para mostrar paso a paso cómo interactúan los usuarios y las distintas partes del sistema durante el proceso de préstamo. De esta forma se puede ver claramente el flujo de mensajes, qué hace cada componente y cómo se desarrolla la acción dentro de la aplicación.



Explicación

Presenta la secuencia de acciones que sucederían para que una acción con cierto fin suceda.

En este caso, un usuario ha solicitado un préstamo para tener acceso y poder leer un libro. Para que el préstamo se lleve a cabo, tiene que suceder ciertas acciones primero, no simplemente se solicita y ya.

Primero, el usuario solicita el préstamo, la interfaz web lo recibe y envía una petición al controlador, éste consulta las penalizaciones del usuario y recibe un número que representa la cantidad de penalizaciones.

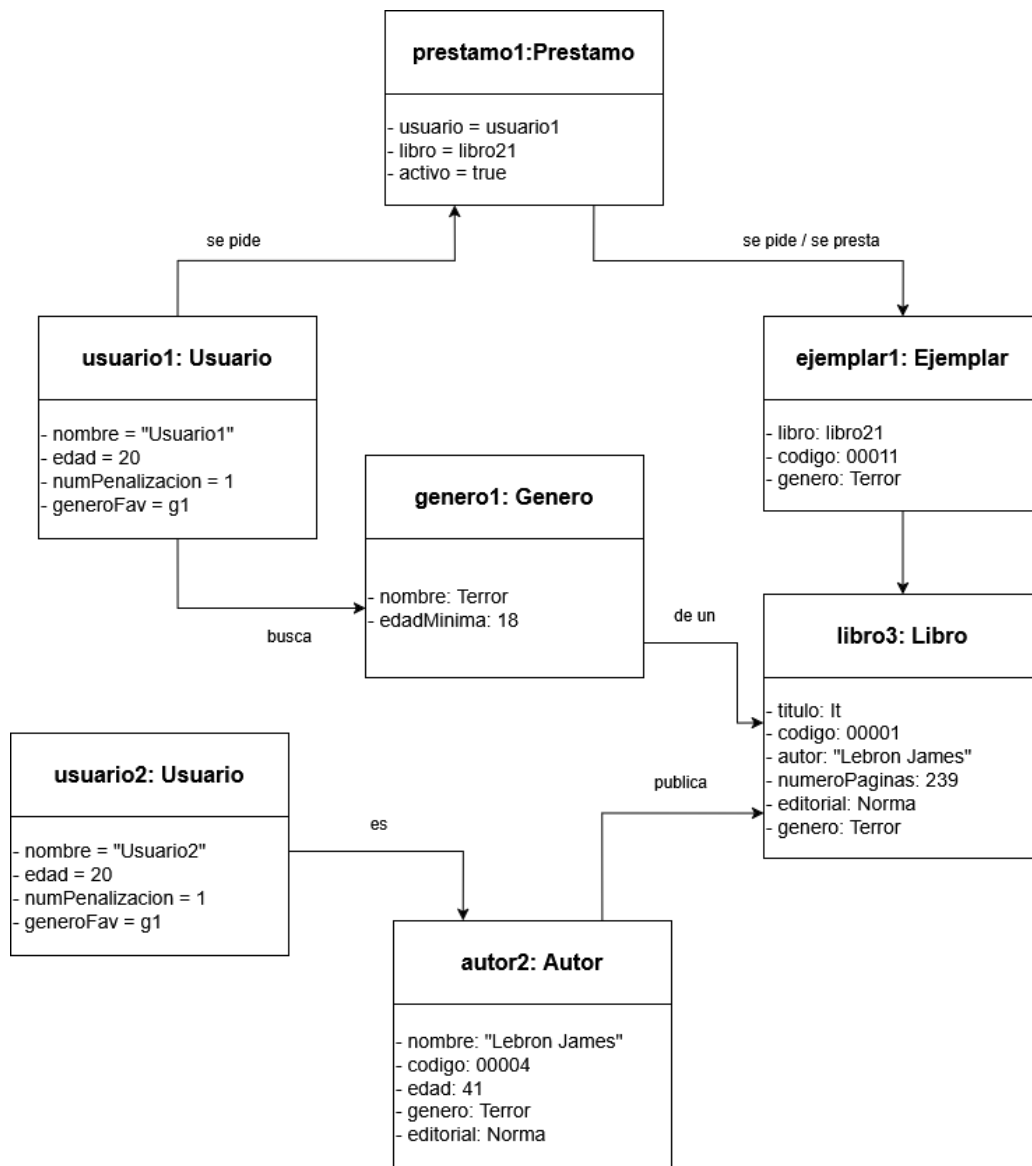
Si el número de penalizaciones es igual o superior al máximo permitido (3 penalizaciones), entonces el controlador le enviará a la web que el préstamo se ha denegado por las penalizaciones, y este mostrará un mensaje para avisar al usuario.

Si el número de penalizaciones es inferior, se consulta si el número de libros prestados es mayor al máximo permitido (3 libros), si el número de libros es igual o mayor, se envía información a la web de que el préstamo ha sido denegado, y esta le pone un mensaje al usuario para avisar.

Si el número de libros es inferior, ya se han terminado todas las verificaciones, por lo que el controlador procede a crear el préstamo e incrementar la cantidad de libros prestados del usuario, así como se devuelve la confirmación, y se muestra un mensaje en pantalla que avisa al usuario que su préstamo se ha realizado con éxito.

Diagrama de objetos

Los diagramas de objetos representan un instante concreto del funcionamiento de nuestro sistema de biblioteca, mostrando instancias reales de las clases y los valores que contienen en un momento determinado. Mientras que el diagrama de clases describe la estructura general del sistema, el diagrama de objetos permite visualizar ejemplos concretos como usuarios, libros, ejemplares o préstamos ya creados, así como las relaciones que se establecen entre ellos dentro de la biblioteca.





Explicación

Representa un escenario real de la ejecución del programa, muestra cómo los distintos objetos se relacionan entre sí.

En el caso de nuestra biblioteca, este es uno de muchos posibles escenarios.

En este escenario, el usuario2 se registró, se convirtió en autor y publicó un libro, que el usuario 1 va a “comprar”.

Por otro lado, el usuario1 busca los géneros de los libros para ver si alguno le interesa. Después de ver un libro que es de su género favorito, pide un préstamo para poder leerlo. El sistema comprueba si supera las condiciones para pedir un préstamo, edad y penalizaciones, y como cumple ambas, seguimos con el préstamo. Para que el préstamo suceda, se crea una copia del libro original en el objeto “ejemplar” y a través del préstamo se le cede al usuario.



Enlace al sistema de versiones

